

NATIONAL UNIVERSITY

College of Computing and Information Technologies

FINAL PROJECT DOCUMENT

FinTrack Stark

Personal Finance, Bill Reminder, and Payment Tracking Integration System

Approved Project Theme No. 8 — Mobile and Web Application Track

Course Code: [ILAGAY DITO]

Course Name: Systems Integration and Architecture

Section: [ILAGAY DITO] Group No.: [ILAGAY DITO]

Group Name: Project Stark

Members

Cain, Akhemee Yujinn

Dabasol, Arvy Karlson

Dela Cruz, John Paul

Heray, Neezar Ivan

Libaton, Jerico

Instructor: [ILAGAY DITO]

Date of Submission: [ILAGAY DITO]

Table of Contents

1. Introduction	1
1.1 Background of the Project	1
1.2 Problem Statement.....	2
1.3 Project Objectives.....	2
1.4 Scope and Limitations.....	3
1.5 Target Users	4
2. Current-State Analysis	5
2.1 Current-State Process	5
2.2 Current-State Problems	5
2.3 Stakeholder Analysis	6
3. Requirements Analysis	7
3.1 Functional Requirements.....	7
3.2 Nonfunctional Requirements	8
3.3 Interface and Data Exchange Requirements	8
3.4 Integration Requirements Traceability Matrix	9
4. Enterprise Architecture.....	11
4.1 Business Architecture	11
4.2 Data Architecture	11
4.3 Application Architecture.....	12
4.4 Technology Architecture.....	12
5. Target-State Architecture	14
5.1 Target-State Process	14
5.2 Target-State Architecture Diagram.....	15
5.3 Architecture Style Selection.....	16
5.4 Integration Pattern Selection	16
5.5 Architecture Decision Record	17
6. System Design.....	18
6.1 Use-Case Diagram.....	18
6.2 Context Diagram.....	19
6.3 Data-Flow Diagram	20
6.4 Sequence Diagram.....	21
6.5 Database Design	22
6.6 User Interface Design	25
7. Integration Design and Implementation	34
7.1 Integration Scenario	34
7.2 API / Workflow / Webhook / Messaging Components.....	34
7.3 Data Mapping Table.....	37
7.4 Sample Payload or Data Format	39
7.5 Validation and Error Handling	42
7.6 Integration Logs	43
8. Security, Governance, and Risk Management	45
8.1 Role-Based Access Control Matrix	45
8.2 Least Privilege and Separation of Duties	45
8.3 Data Privacy and Protection	46
8.4 Audit Trail	46
8.5 Integration Ownership and Governance	47
8.6 Risk Register.....	48
9. Testing and Evaluation.....	49
9.1 Test Plan	49
9.2 Functional Test Cases	49

9.3 Integration Test Cases	50
9.4 Error-Handling Test Cases.....	50
9.5 End-to-End Test Scenario.....	51
9.6 Test Results Summary.....	51
10. Deployment, Backup, and Recovery	53
10.1 Deployment Environment	53
10.2 Deployment Evidence	53
10.3 Backup Plan.....	54
10.4 Recovery Plan	55
11. System Demonstration Guide.....	56
11.1 Demo Script	56
11.2 Demo Accounts.....	56
11.3 Sample Data.....	57
12. Limitations and Future Enhancements.....	58
12.1 System Limitations.....	58
12.2 Recommended Enhancements	58
13. Individual Contribution	59
14. References	60
15. Appendices	61
Appendix A — Complete API Endpoint List	61
Appendix B — Additional Screenshots	62
Appendix C — Complete Test Results.....	67
Appendix D — Source Code Excerpts	69
Appendix E — Project Folder Structure	70

1. Introduction

1.1 Background of the Project

Filipino households manage a steady stream of recurring obligations: electricity (Meralco), water (Maynilad, Manila Water), internet and mobile plans, rent, credit cards, and a growing number of subscriptions. These are paid through many different channels — payment centres, bank apps, and e-wallets such as GCash and Maya — and each channel keeps its own record, if it keeps one at all. Income arrives just as irregularly: salary, freelance work, allowances.

In practice the information that a person needs to stay on top of their money is scattered: due dates sit in SMS messages and paper bills, payments sit in separate apps, receipts sit in a drawer or a phone's photo gallery, and totals sit in a notebook or a spreadsheet that is updated by hand when there is time. Nothing connects these pieces, so a missed due date, a typing mistake, or a budget that has already run out is usually discovered too late.

FinTrack Stark is an integrated web and mobile (installable Progressive Web App) system that brings these pieces together. It connects the user's wallet records, bill reminders, payment status, budget tracking, a receipt review workflow, a notification system and a financial dashboard — the integration focus named for Approved Project Theme No. 8. The project's emphasis, as the specification requires, is on how these modules, users and services are connected: a payment automatically becomes an expense entry, a notification and an audit record; a separate reminder service raises alerts on its own schedule through a shared database; a submitted receipt automatically enters a review queue and every decision is pushed to the people concerned as it happens.

How the specification's terms map to this project

The project specification is written for a creative-production pipeline (projects, assets, artists, renders). Because the approved theme is a personal-finance system, each generic component is implemented with its finance equivalent. Table 1 is used throughout this document.

Table 1. Mapping of the specification's generic components to FinTrack Stark

Specification term	FinTrack Stark equivalent	Where in the system
Project / Production Module	Bill categories (Utilities, Housing, Internet, Credit, Subscription, Other) and the bills inside each	Bill Categories, Category Detail, Bill Reminders
Project List / Project Details	List of bill categories with totals / one category with its bills and statuses	/categories, /categories/:name
Asset / File Submission	A receipt or proof of payment (photo, PDF, or Google Drive / OneDrive / Dropbox link) attached to an income or expense entry	Log Income/Expense, My Entries
Task status	Bill status (Upcoming, Due Today, Overdue, Paid) and receipt status (For Review, Verified, Needs Revision, Rejected)	Bills, My Entries, Receipt Review
Version Tracking (v1, v2, revised, approved, rejected, final)	Entry versions (v1 → v2, Earlier version, Deleted) and receipt versions (v1 Needs Revision → v2 Verified)	Revision History, Receipt dialog
Review and Approval Workflow	An Admin verifies, rejects or requests a revision of each submitted receipt	Receipt Review
Comment / Feedback Module	Personal notes and notes on entries, including the Admin's review decision notes	Notes / Feedback

Specification term	FinTrack Stark equivalent	Where in the system
Reviewer / Director	Admin role (reviews receipts; keeps no wallet of their own)	Admin Console, Receipt Review
Artist, Animator, Client	User (member) — records their own money and submits receipts	Home dashboard
Admin / Project Manager	Admin role — receipt review, accounts, roles, password resets, system settings, audit log	Admin Console, System Settings

1.2 Problem Statement

Individuals and households who pay several recurring bills through different channels have no single, reliable record of what is due, what has been paid, what each peso was spent on, and whether the records are backed by proof. The current, manual way of working causes:

- **Missed and late payments** — due dates are remembered, or not; there is no reminder that arrives before the due date, so penalties and disconnections happen.
- **Double work and transcription errors** — every payment is made in one place and then copied by hand into a notebook or spreadsheet, where amounts are mistyped and corrections overwrite the original with no history.
- **Scattered, unverifiable proof** — receipts and payment screenshots are kept in drawers and galleries; nobody checks that a recorded figure matches its receipt.
- **No early warning on spending** — budgets, if any, are checked at the end of the month, after they have already been exceeded.
- **No accountability trail** — there is no record of who recorded, changed, or deleted what and when.

The project addresses the question: *how can bill reminders, payment records, income and expense entries, receipts, budgets and notifications be integrated into one system so that payments are recorded once, reminders and warnings arrive on time, every figure can be traced and verified, and every action is accounted for?*

1.3 Project Objectives

General objective

To design, develop, integrate, deploy and defend a web and mobile (PWA) personal-finance system that connects bill reminders, payment tracking, income and expense records, receipt submission and review, budgets, notifications and a financial dashboard, using appropriate architecture, integration patterns, security controls, logging, testing and documentation.

Specific objectives

1. Identify the problems of the current manual bill-and-receipt process and analyse its current and target states (Sections 2 and 5).
2. Identify the stakeholders, systems, interfaces and data exchanged between them (Sections 2.3 and 3.3).
3. Design the business, data, application and technology architecture (Section 4).
4. Select and justify an architecture style (layered, with event-driven elements) and integration patterns (point-to-point REST, shared database, event-driven messaging) (Section 5).

5. Develop a working, deployed prototype with role-based dashboards for User and Admin (Sections 6 and 10).
6. Implement integration components: the bill-payment workflow cascade, the receipt review workflow, live event messaging, the scheduled reminder service and external-storage links (Section 7).
7. Apply validation, error handling, role-based access control, least privilege and separation of duties (Sections 7.5 and 8).
8. Produce audit and integration logs that record successful and failed actions with a reference to the record involved (Sections 7.6 and 8.4).
9. Perform functional, integration, error-handling, security and end-to-end testing with automated, repeatable tests (Section 9).
10. Prepare backup, recovery and deployment evidence (Section 10).

1.4 Scope and Limitations

In scope

- Account registration, sign-in and two roles (User, Admin) with role-specific dashboards.
- Bills organised by category, with due dates, monthly repetition, payment recording and payment history.
- Automated bill reminders (three days before, on the due date, and when overdue) from a scheduled reminder service.
- Income and expense entries that count immediately, with edit-as-new-version and soft delete (Revision History).
- Receipt / proof-of-payment submission (upload up to 2 MB or cloud link), versioning, and review by an Admin.
- Monthly budgets per category with warnings at 80% and 100%.
- Notes and feedback, an in-app notification log, live updates without refreshing, and reports.
- Audit and integration log with Success/Failed status; user, role and password administration; system settings (reminder lead time, budget warning level, session length) managed by the Admin.
- Responsive web interface installable on phones as a Progressive Web App; cloud deployment; backup and restore.

Out of scope / limitations

- No real payment processing: the system records payments made elsewhere and never connects to banks or e-wallets (specification §21.7).
- No e-mail or SMS delivery: notifications are in-app and live; password recovery is done by an Admin-issued temporary password.
- No native Android/iOS application: the mobile experience is an installable PWA (the specification allows "mobile and/or web").
- No automatic reading (OCR) of receipt contents: an Admin compares the receipt with the entry.
- Single currency (Philippine peso) and single time zone (Asia/Manila).
- Sample data only; no real personal or financial data is used (specification §21.8).

1.5 Target Users

Table 2. Target users and roles

Role	Who they are	What they do in the system
User	Any individual or household member managing their own bills and money (students, young professionals, families).	Records bills, payments, income and expenses; attaches receipts; sets budgets; writes notes; reads their dashboard, reports and notifications.
Admin	The person responsible for operating the system and checking evidence — for example a household head, the treasurer of a shared fund, or a finance officer.	Reviews submitted receipts against the entries they prove (verify, reject, request revision); manages accounts and roles, resets passwords, sets the system settings, monitors the Admin Console and the audit log. Keeps no wallet of their own, and sees only the receipts submitted for review and the one entry each proves — never a whole wallet.
Reminder Service (system actor)	An automated background process.	Checks unpaid bills every day and raises reminders and overdue alerts.

2. Current-State Analysis

2.1 Current-State Process

Before integration, the process for a typical bill is entirely manual and spread across disconnected tools (Figure 1): a bill arrives on paper, by SMS or by e-mail; the person writes the due date down somewhere; whether it is paid on time depends on memory; payment is made at a payment centre, a bank app or an e-wallet; the receipt or screenshot is kept wherever it lands; the amount is copied by hand into a notebook or spreadsheet; and at the end of the month, if at all, spending is added up by hand. Income is recorded the same way. No one checks the copied figures against the receipts.

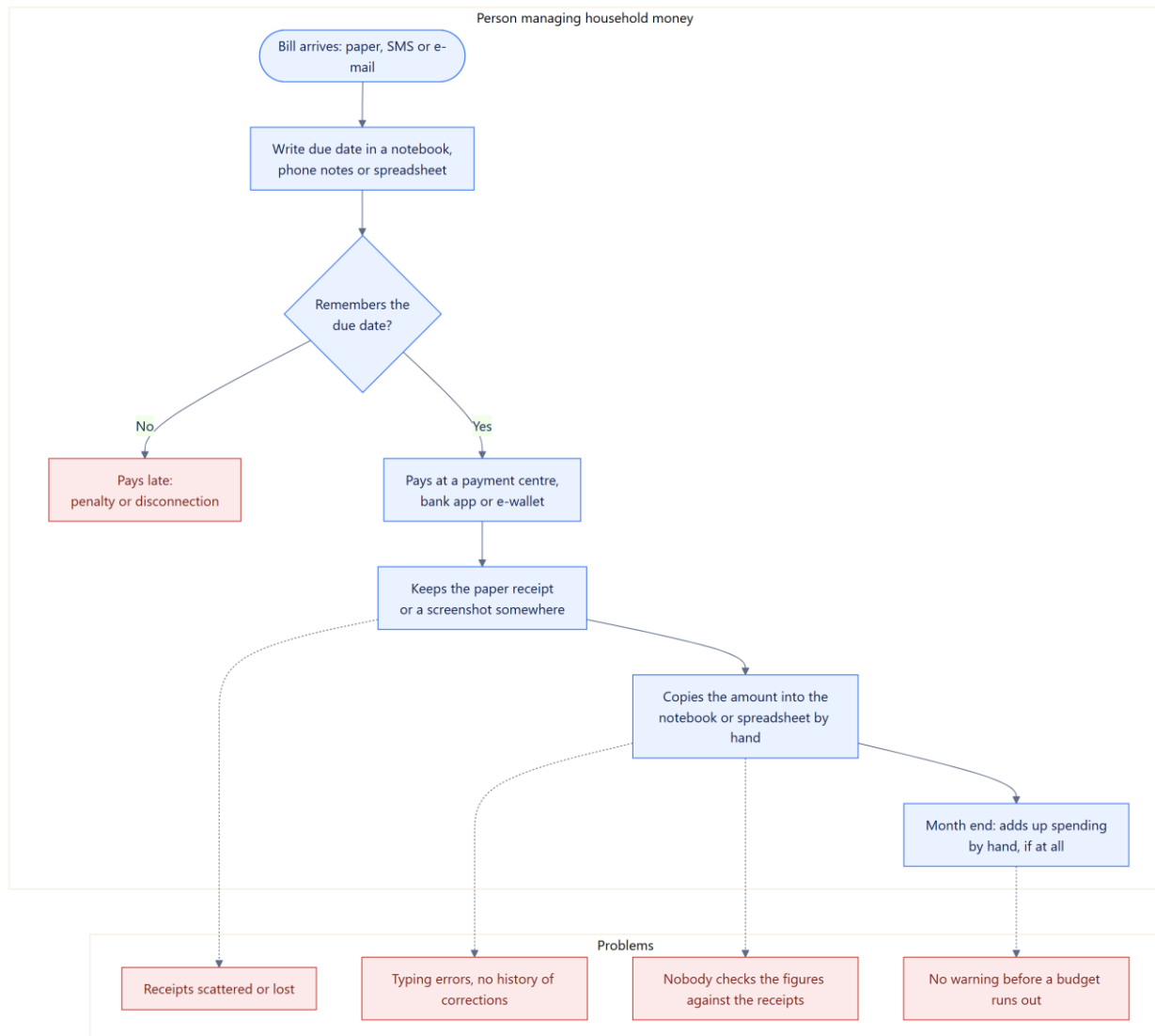


Figure 1. Current-State Process Flow (before integration)

2.2 Current-State Problems

Table 3. Current-state problems and how the target system addresses them

Problem	Effect	Addressed in FinTrack Stark by
Manual tracking of due dates	Late payments, penalties, disconnections	Reminder service: alerts 3 days before, on the day, and when overdue (FR-011)

Problem	Effect	Addressed in FinTrack Stark by
Payments recorded twice (pay, then copy)	Double work; forgotten or mistyped entries	Mark Paid creates the expense entry automatically (workflow automation, FR-007)
Scattered receipts	Lost proof; disputes cannot be settled	Receipts attached to the entry they prove, as an upload or a cloud link (FR-003)
No checking of figures against receipts	Errors and false entries go unnoticed	Receipt Review: an Admin verifies, rejects or requests a revision (FR-005)
Corrections overwrite the original	No history; changes cannot be explained	Editing creates v2 and keeps v1; deleting is a soft delete (FR-004, FR-015)
Budgets checked after the month ends	Overspending discovered too late	Budget alerts at 80% and 100% as each expense is recorded (FR-012)
Duplicated and inconsistent versions	Two different totals for the same month	One shared database; one current version per entry; atomic updates prevent duplicates (NFR-010)
Unclear responsibility	Nobody knows who changed what	Audit log of every action with user, time, status and record reference (FR-010)

2.3 Stakeholder Analysis

Table 4. Stakeholders

Stakeholder	Interest / need	Involvement in the system	Influence
Users (members)	Never miss a due date; know where money goes; keep proof of payments	Primary users of every finance screen	High
Administrator	Accounts that work, correct roles, visibility of what happens, and a clear queue of receipts with the entry each one must prove	Admin Console, Receipt Review, User & Role Management, Audit Log	High
Project team (developers)	A maintainable, testable, defensible system	Builds, tests, deploys and documents the system	High
Instructor / technical panel	Evidence that integration, security and testing requirements are met	Evaluates the prototype, document and defense	High
Billers, banks and e-wallets	Paid on time (indirect)	External: payments happen there and are recorded here	Low
Cloud providers (Render, MongoDB Atlas)	Service terms, resource limits	Host the application and the database	Medium
Cloud storage (Google Drive, OneDrive, Dropbox)	—	Hold receipt files that are attached by link	Low

3. Requirements Analysis

3.1 Functional Requirements

Table 5. Functional requirements (16)

ID	Requirement	Actor	Priority
FR-001	The system must allow people to register and authorized users to log in, and must give each role its own dashboard.	All	Must
FR-002	The system must allow users to create, edit and delete bills, organised by category (the "project" records).	User	Must
FR-003	The system must allow users to record income and expense entries and to submit a receipt for an entry as an uploaded file (JPG, PNG, WEBP, PDF) or a cloud file link.	User	Must
FR-004	The system must track versions of entries (v1, v2 ...) and of receipts, keeping earlier versions.	System	Must
FR-005	The system must allow the Admin (reviewer) to verify, reject, or request a revision of a submitted receipt.	Admin	Must
FR-006	The system must store notes and feedback, per entry and personal, including the Admin's review decision notes.	User, Admin	Must
FR-007	The system must automatically update statuses after submission, payment or review (receipt → For Review; bill → Paid with an expense entry; next monthly bill scheduled).	System	Must
FR-008	The system must record notification logs for important workflow events and deliver them to the addressed user without a page refresh.	System	Must
FR-009	The system must display dashboard summaries (balance, spending by category, bills, budgets; review queue; system activity).	All	Must
FR-010	The system must record audit logs for important user actions, including failed and refused actions.	System	Must
FR-011	The system must remind users of unpaid bills before the due date, on the due date and when overdue, without duplicate alerts.	Reminder Service	Must
FR-012	The system must let users set monthly budgets per expense category and warn at a set level (80% by default) and at 100% use.	User	Should
FR-013	The system must allow the Admin to manage user roles, delete accounts and issue temporary passwords; an Admin keeps no wallet of their own.	Admin	Must
FR-014	The system must provide reports of spending by category, budgets and bill payments.	User	Should
FR-015	The system must allow users to correct and delete their own entries without losing the history.	User	Must
FR-016	The system must let the Admin manage system settings — reminder lead time, budget warning level and session length — with validation, effect without redeploying, and an audit record of every change.	Admin	Should

3.2 Nonfunctional Requirements

Table 6. Nonfunctional requirements (11)

ID	Category	Requirement	How it is met / measured
NFR-001	Security	Only authenticated users can access records.	Every /api route except login, register and health requires a valid session token (ST-01).
NFR-002	Authorization / Privacy	Users can only access their own records; receipts are visible only to their owner and the Admin who reviews them; an Admin keeps no wallet.	Owner checks in every controller; finance endpoints refuse an Admin; the Admin sees a receipt and its one entry only (ST-02, ST-04, RT-05, RT-31).
NFR-003	Auditability	All approval, rejection, revision, payment, edit and deletion actions — and failures — must be recorded in audit logs.	Audit log with status and record reference (FT-08, ET-07, IT-07).
NFR-004	Reliability	The system must validate required fields and data types before saving or sending data.	Server-side validation on every write; schema limits (ET-01, ET-02, RT-04, RT-30).
NFR-005	Usability	Dashboards must be understandable; screens must work on phones from 320 px wide.	Role dashboards; bottom tab bar on phones; no sideways scrolling (RT-13, RT-14, RT-33).
NFR-006	Error handling	The system must give clear error messages for failed submissions and keep what was typed.	Plain-language 4xx messages; forms kept on failure (ET-01–ET-07, RT-11).
NFR-007	Availability / Recoverability	Data and receipt files must be backed up and restorable.	npm run backup / restore including receipt files (RT-34; Section 10).
NFR-008	Maintainability	Code and documentation must be clear enough for technical review.	Layered folders (routes, middleware, controllers, services, models); this document; README.
NFR-009	Performance	A screen must refresh with one request; a change must reach open screens within about a second.	GET /api/sync; Server-Sent Events (RT-20, IT-02); typical API calls 20–40 ms locally (Section 7.4).
NFR-010	Integrity / Concurrency	Simultaneous actions must not create duplicates (double payment, double decision, two v2s).	Atomic conditional updates; tested with 10 simultaneous requests (ET-05, RT-02, ET-07).
NFR-011	Session security	Passwords and session tokens must never be stored in readable form; sessions must expire.	bcrypt password hashes; SHA-256 token hashes; expiry set by the Admin (8 hours by default, 1–24); rate-limited login (RT-06, RT-37).

3.3 Interface and Data Exchange Requirements

Table 7. Interfaces and data exchanged between modules and systems

From → To	Data	Format / channel	Trigger
Web client → API	Credentials; bills; entries; receipts (base64 file or URL); budgets; notes; review decisions	HTTPS, REST, JSON; Bearer token	User action
API → Web client	All of the signed-in person's data in one response	GET /api/sync, JSON	Page load, live event, reconnect

From → To	Data	Format / channel	Trigger
API → Web client (live)	"Something changed" events with a topic (no data)	Server-Sent Events, GET /api/events	Any write that concerns the user or their role
Bill payment workflow → Transactions	Expense entry built from the bill (category, amount, date, note)	Internal service call, MongoDB write	POST /api/bills/:id/pay
Receipt workflow → Notifications / Comments / Audit	Decision, note, version, entry summary	Internal, MongoDB writes	Receipt submitted or decided
Reminder Service ↔ Database	Reads unpaid bills due within 3 days; writes notifications, lastRemindedOn, audit entry	Shared MongoDB database	Daily schedule (08:00) and API start-up
Web client → Cloud storage	Receipt file link (Google Drive, OneDrive, Dropbox)	https URL stored as text	User attaches a link
API ↔ MongoDB Atlas	All persistent data	MongoDB wire protocol over TLS	Every request
Operator → Backup file	Every collection including receipt files	JSON file (base64 for files)	npm run backup / restore

3.4 Integration Requirements Traceability Matrix

Each requirement is traced to the stakeholder who needs it, the module or system that implements it, the diagram that shows it, and the automated test cases that verify it (Section 9).

Table 8. Requirements traceability matrix

Req.	Stakeholder	Module / system	Diagram	Test cases
FR-001	All users	Auth API, sessions, role dashboards	Use-Case, Context	FT-01, RT-08, RT-13
FR-002	User	Bills API, Bill Categories	Use-Case, DFD 2.0	FT-02, FT-09
FR-003	User	Transactions API, Receipts API	DFD 3.0/4.0, Sequence	FT-03, FT-11, ET-06
FR-004	User, Admin	Transaction & Receipt versioning	ERD, Receipt states	FT-04, IT-07, RT-19
FR-005	Admin	Receipt Review workflow	Sequence, Target-State	FT-12, IT-07, E2E-02
FR-006	User, Admin	Comments API, Notes screen	DFD D7	FT-06, RT-27, RT-28
FR-007	User	Payment cascade, receipt automation	Payment cascade, Target-State	IT-01, IT-07, RT-17
FR-008	User, Admin	Notifications, event bus (SSE)	Application, Sequence	IT-02, IT-03, RT-29
FR-009	All	Dashboards (Home, Review Desk, Admin Console)	Application	FT-07, RT-13
FR-010	Admin, panel	Audit service, failure log	DFD 6.0	FT-08, ET-07
FR-011	User	Reminder worker + in-API sweep	Deployment, DFD 5.0	IT-06, RT-23
FR-012	User	Budgets, budget alerts	DFD 3.0	IT-05, RT-18
FR-013	Admin	Users API	Use-Case	ST-03, RT-06
FR-014	User	Reports screen	Application	E2E-01, IT-05
FR-015	User	Transactions API (versions, soft delete)	ERD	FT-05, RT-01
FR-016	Admin	Settings API; reminder service, budget alerts, sessions	Use-Case, ERD	RT-36, RT-37

Req.	Stakeholder	Module / system	Diagram	Test cases
NFR-001/002	All users	requireAuth, owner checks, RBAC	Context	ST-01, ST-02, ST-04, RT-31
NFR-003	Admin, panel	Audit log	DFD 6.0	FT-08, ET-07, IT-07
NFR-004/006	All users	Controllers, schemas, central error handler	Payment cascade	ET-01–ET-07, RT-30
NFR-007	Admin	backup.js, restore.js	Deployment	RT-34
NFR-010	All users	Atomic claims	Sequence, Payment cascade	ET-05, RT-02, ET-07

4. Enterprise Architecture

4.1 Business Architecture

The system supports eight business processes. Each has a single owning module, a trigger and a defined output, which is what makes them integrable: the output of one process is the trigger of another (for example, a payment produces the expense entry that the budget process then checks).

Table 9. Major business processes

Process	Actor	Trigger	Output
BP1 Account and access management	User, Admin	Registration, sign-in, role change, password reset	Account, session token, role-specific dashboard, audit entry
BP2 Bill lifecycle	User, Reminder Service	Bill added; daily schedule; Mark Paid	Reminders, Paid bill, expense entry, next monthly bill
BP3 Income and expense recording	User	Entry saved, edited or deleted	Counted entry, new version, soft-deleted entry
BP4 Receipt submission and review	User, Admin	Receipt attached; Admin decision	Receipt version with status, decision note, notifications
BP5 Budget monitoring	System	Expense recorded or paid	80% / 100% budget alert
BP6 Notification	System	Any workflow event	Notification log entry; live update to open screens
BP7 Oversight, settings and audit	Admin	Any action, success or failure; a settings change	Audit/integration log; Admin Console activity; new reminder, budget and session behaviour
BP8 Backup and recovery	Operator (Admin)	Before a deployment, weekly, or after a failure	JSON backup; restored database

4.2 Data Architecture

All persistent data lives in one MongoDB database (ten collections). For each entity there is exactly one **source of truth** — the module allowed to create and change it — which is the rule that keeps the shared-database integration safe (Section 5.4).

Table 10. Data entities and their source of truth

Entity (collection)	Holds	Source of truth (writer)	Key rules
User (users)	Name, email, password hash, role	Auth API (self), Users API (Admin)	Email unique; role ∈ Admin, User
Session (sessions)	SHA-256 hash of the token, role copy, expiry	Auth API	Expires after 8 hours (TTL index)
Bill (bills)	Name, category, amount, due date, paid state, repetition	Bills API; Reminder Service owns only <code>lastRemindedOn</code>	Paid bills cannot be edited or deleted
Transaction (transactions)	Income/expense entries and their versions	Transactions API; payment workflow	One current version per chain; never hard-deleted
Receipt (receipts)	Uploaded file or link, version, review status	Receipts API (owner submits, Admin decides)	≤ 2 MB; type checked; one decision per version
Budget (budgets)	Monthly limit per category	Budgets API	Unique per user and category

Entity (collection)	Holds	Source of truth (writer)	Key rules
Comment (comments)	Notes and decision notes	Comments API; receipt review	Editable only by its author
Notification (notifications)	Message addressed to one user	Notification service; Reminder Service	Readable only by the addressee
AuditLog (auditlogs)	Who, what, when, status, detail, record reference	Audit service, failure-log middleware, Reminder Service	Append-only through the application
Setting (settings)	Reminder lead time, budget warning level, session length	Settings API (Admin); read by the API and the reminder service	One document; every value within its limits

4.3 Application Architecture

FinTrack Stark is made of three applications that interact: the **web client** (a React single-page application that installs as a PWA), the **API server** (Node.js and Express, organised in layers), and the **reminder service** (a separate Node.js process with its own schedule). The database is shared. The table below lists the modules; the architecture diagram in Section 5.2 shows how they interact.

Table 11. Application modules

Module	Responsibility	Main interactions
Authentication & sessions	Register, log in, log out, profile, password change, 8-hour sessions	Every other module (requireAuth)
Bills & payments	Bills by category, Mark Paid cascade, monthly repetition	Transactions, Notifications, Audit, Budgets
Entries (transactions)	Income and expenses, versions, soft delete	Budgets, Receipts, Comments
Receipts & review	File/link submission, versions, Admin decisions	Transactions, Comments, Notifications, Audit, events
Budgets & alerts	Monthly limits, 80% / 100% warnings	Notifications
Notes / feedback	Personal notes and notes on entries	Transactions, Receipts
Notifications & live events	Notification log; Server-Sent Events to open screens	All writing modules
Audit & integration log	Success and failure records with reference ids	All modules; Admin Console
Administration	Roles, account deletion, temporary passwords	Sessions, Audit, Notifications
System settings	Reminder lead time, budget warning level, session length	Reminder service, Budget alerts, Sessions, Audit
Reminder service	Daily sweep of unpaid bills	Bills, Notifications, Audit (via shared database)
Sync	One response with everything a screen shows	All read models

4.4 Technology Architecture

Table 12. Technology components

Component	Technology (version)	Purpose / reason
Frontend	React 18.3, React Router 7.18, Vite 5.4	Single-page app; every screen has its own address; fast production build

Component	Technology (version)	Purpose / reason
Mobile	Progressive Web App (Web App Manifest + service worker)	Installs on Android and iPhone home screens; opens offline; no app store needed
Backend / API	Node.js 24, Express 4.22	REST JSON API, static file serving, Server-Sent Events
Data access	Mongoose 8.24 (MongoDB driver 6.20)	Schemas, validation, atomic conditional updates
Database	MongoDB Atlas M0 (AWS Hong Kong), replica set of 3 nodes	Managed cloud database; TLS; least-privilege database user
File storage	Receipt files in MongoDB (≤ 2 MB); links to Google Drive / OneDrive / Dropbox	No separate storage service needed; external storage simulated by links
Scheduler / worker	node-cron 4.6 (separate process); in-API daily sweep	Bill reminders on a schedule
Notification service	In-process event bus + Server-Sent Events; notification collection	Live, in-app notifications without polling
Security	Node crypto (scrypt, SHA-256, randomBytes), hand-written rate limiter and security headers	Password hashing, session tokens, brute-force protection, CSP/HSTS
Hosting	Render Web Service (Singapore), HTTPS via Render's edge	Runs API and built frontend as one service; builds from GitHub
Testing	Playwright 1.63 with Chrome	68 automated tests against an isolated test database
Source control	Git, GitHub (jeypydc02-ui/SIA-Final-Project)	Version history, deployment source, code backup
Dashboard	Role dashboards built into the client (Home, Review Desk, Admin Console)	Summaries, activity, review queue

5. Target-State Architecture

5.1 Target-State Process

The figure below shows the improved, integrated workflow. A bill is entered once; the reminder service warns the user before it is due; marking it paid triggers a cascade (expense entry, notification, audit record, next monthly bill and a budget check) with no retyping. Any income or expense can carry a receipt; attaching one automatically puts it in the review queue and notifies the Admins, whose decision is returned to the owner as a notification and a note. Each step is written to the audit log with its status, and every open screen updates by itself.

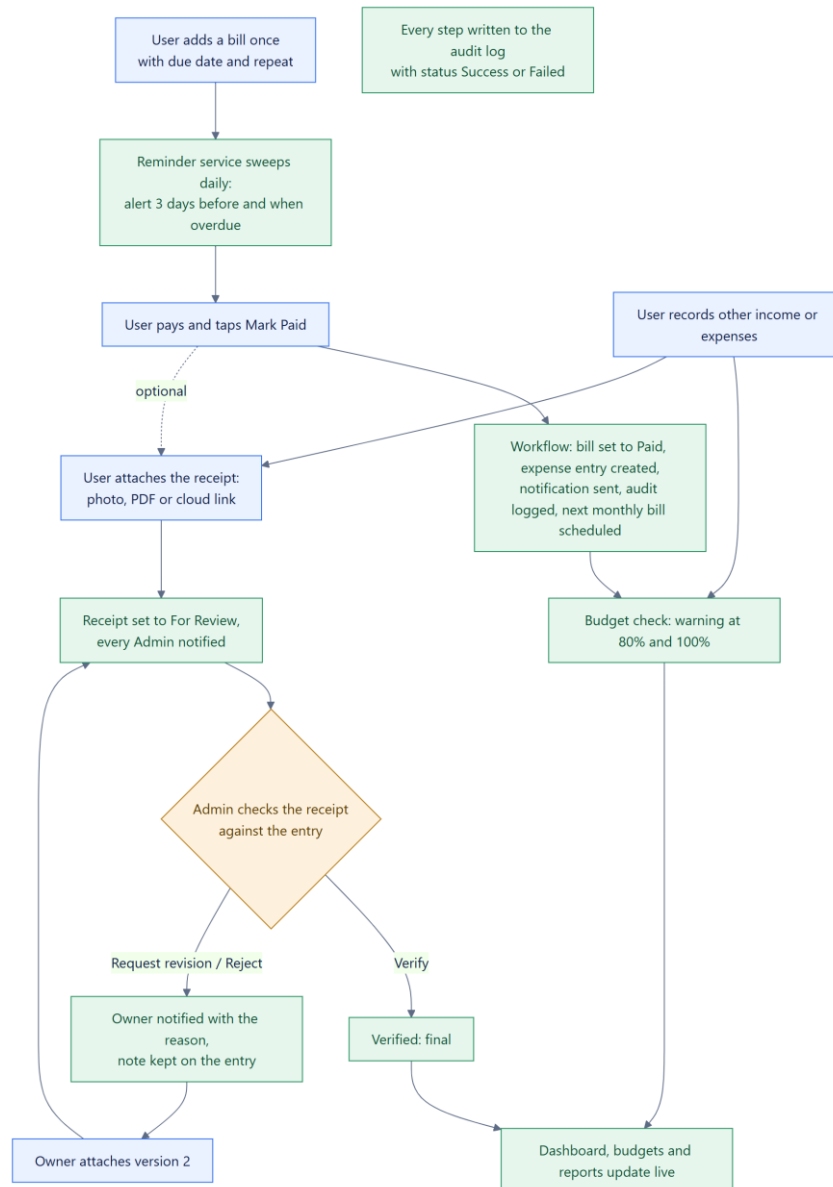


Figure 2. Target-State Process Flow (after integration)

5.2 Target-State Architecture Diagram

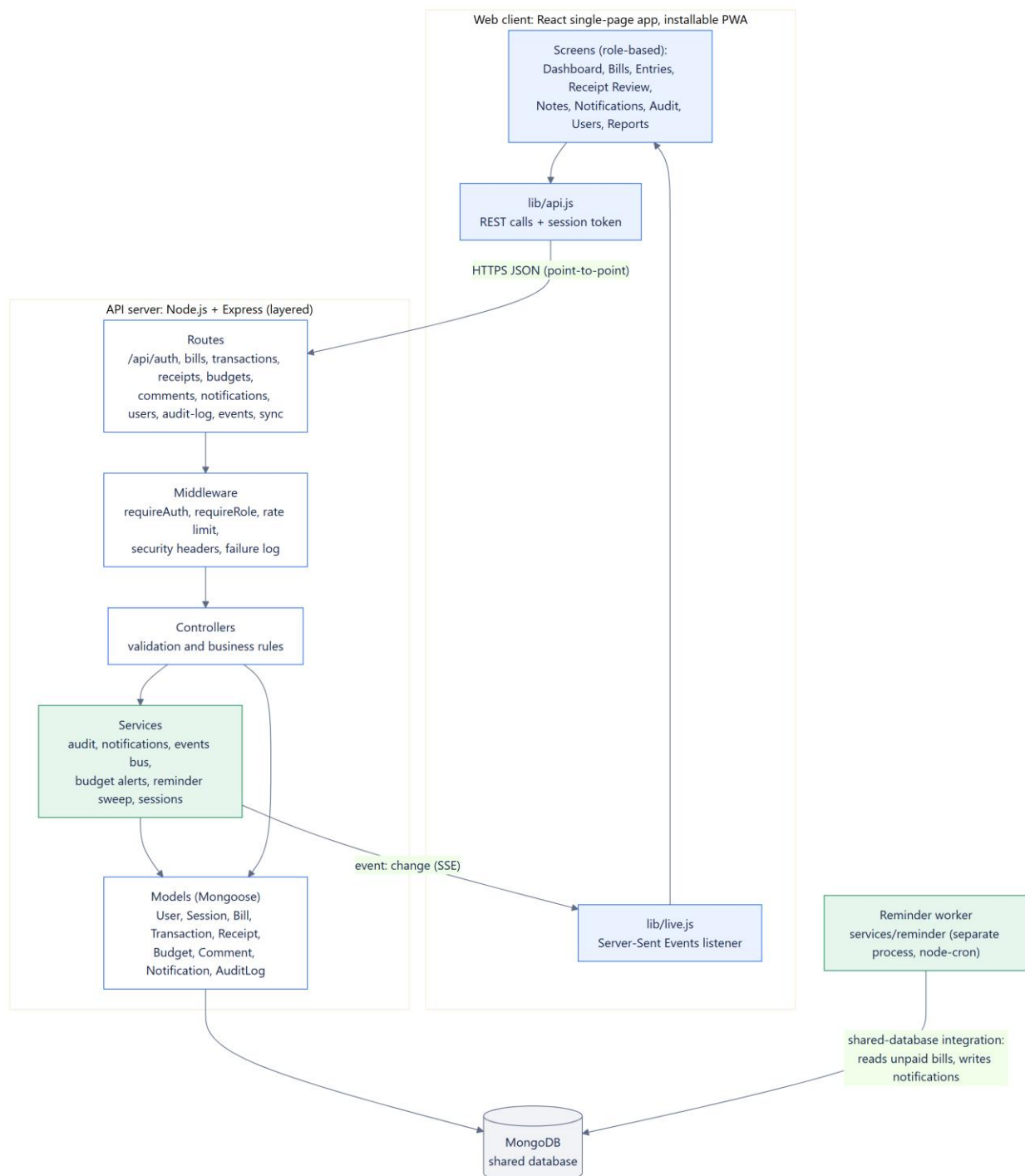


Figure 3. Target-state / application architecture: modules and how they are connected

The client talks to the API point-to-point over HTTPS (REST/JSON). Inside the API, a request passes through routes, middleware (authentication, role checks, failure logging), controllers (validation and business rules), services (audit, notifications, events, budget alerts) and models. The services publish *change* events on an in-process bus, which the events route streams to open browsers. The reminder worker is a separate process that integrates through the shared database.

5.3 Architecture Style Selection

Table 13. Comparison of architecture styles

Style	Strengths for this project	Weaknesses for this project	Decision
Layered	Clear separation of UI, business logic and data access; easy to explain, test and maintain for a five-person team; one deployable unit fits free hosting	Layers can become tightly coupled if rules leak into routes	Selected (primary)
Service-Oriented (SOA)	Reusable services (notification, audit, reporting)	Needs a service contract/registry and network calls between services; heavy for one database and one team	Partly adopted: services are internal modules (audit, notifications, events)
Microservices	Independent deployment and scaling per service	Several deployables, network failures between them, distributed data; free hosting allows one web service; not justified by the load	Not selected
Event-Driven	Upload, review, payment and reminder are naturally events that trigger responses; enables live updates	Harder to trace if used for everything; needs a broker for multiple processes	Selected (secondary): in-process event bus + SSE for notifications and live updates

Justification. A layered architecture keeps each concern in one place — routes only map URLs, middleware enforces access, controllers hold the rules, services perform cross-cutting work, and models own the data — which made it possible to add the receipt review module and the integration log without touching unrelated code. Event-driven elements are used exactly where the domain is event-shaped: a payment, a submitted receipt, a decision or a reminder *happens*, and other parts react (notification, live refresh of open screens, audit). Microservices were rejected because the system's scale does not need independent scaling, and the free hosting tier supports a single web service.

5.4 Integration Pattern Selection

Table 14. Comparison of integration patterns

Pattern	Where it would fit	Risk / cost	Decision
Point-to-Point	Web client ↔ API: one consumer, one provider, a clear REST contract	Becomes hard to maintain with many systems — not the case here (two)	Used for client ↔ API
Hub-and-Spoke	A central broker routing data among many systems	Extra infrastructure with only one API and one worker	Not used (no hub needed)
Shared Database	API and reminder service both work on bills and notifications	Risky if data ownership is unclear	Used for API ↔ reminder service, with explicit field ownership and atomic claims
Event-Driven	Workflow events (payment, receipt submitted/decided, reminder) trigger notifications and live updates	Events must not carry private data; ordering	Used inside the API and to clients (SSE); events carry only a topic

Justification. The combination keeps each connection as simple as it can be. The client–API link is point-to-point because there is one client application and one API. The reminder service uses the

shared database because it must work while no one is using the app and even if the API is asleep; its risk is controlled by **ownership**: the reminder service may only set `lastRemindedOn` on a bill and insert notifications and its own audit entry, and it claims each bill with a conditional update (`lastRemindedOn ≠ today`) so two sweeps on the same day can never send the same alert twice (verified in IT-06 and RT-23). Event-driven messaging delivers workflow results to people as they happen; events contain only a topic such as receipts, and the browser then reloads its own data through the authorised API, so no private data travels in an event.

5.5 Architecture Decision Record

Table 15. Architecture decision records

ADR	Decision	Context and rationale	Consequences
ADR-001	Layered modular monolith: Express API + React client, deployed as one service	Five-person team, one database, free single-service hosting; clarity for the defense matters more than independent scaling	+ simple deployment and testing; + one URL. – whole app redeploys for any change; scaling is vertical only
ADR-002	Reminder service integrates through the shared database (not through the API)	Reminders must run on a schedule independent of user activity; the API may be asleep on free hosting	+ works with no user online; + the worker never needs credentials to the API. – field ownership must be respected (documented in 8.5); the API also runs the sweep daily so reminders still happen where no worker is hosted
ADR-003	Live updates with Server-Sent Events, not polling or WebSockets	Updates flow one way (server → browser); SSE works over plain HTTPS through proxies; polling would multiply requests on slow mobile data	+ notifications appear without refresh; + events carry no data. – one open connection per tab (capped at 10 per account); an in-process bus means one API instance
ADR-004	Review the receipt, not the entry; the Admin reviews and keeps no wallet	An earlier version required approval before an entry counted; users found the delay confusing. The specification still requires Asset Submission and Review/Approval. A separate Reviewer role was tried and folded into Admin so that one accountable role oversees the system	+ entries count immediately; + review checks real evidence; + the reviewer never submits evidence (no wallet). – an unverified entry still counts in the balance (shown by its receipt status); – the Admin sees receipts (limited to the receipt and its one entry)
ADR-005	Server-side sessions with hashed tokens instead of stateless JWT	Logout, password change and role change must take effect at once on every device	+ immediate revocation; + a leaked database does not leak working tokens. – one database lookup per request
ADR-007	System settings live in the shared database, not in environment variables	The Admin must be able to change reminder, budget and session behaviour (spec §12) without a redeploy, and the separate reminder worker must use the same values as the API	+ one source of truth for both processes; + every change audited. – values are cached for up to 5 seconds in a process that did not make the change
ADR-006	Store receipt files (≤ 2 MB) in MongoDB, or store a link	No paid object storage; most receipts are photos that compress well	+ backups include the files; + no extra service. – size limit; large files must be attached as a cloud link

6. System Design

6.1 Use-Case Diagram

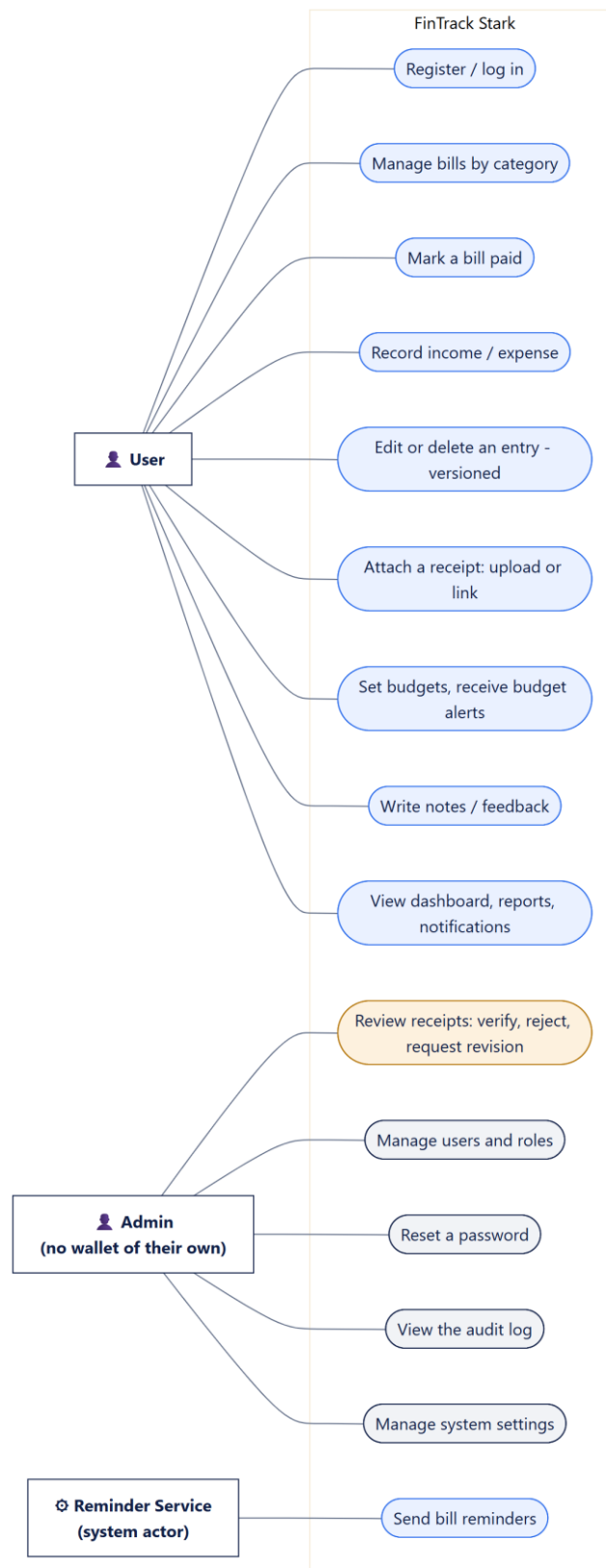


Figure 4. Use-Case Diagram: functions per role

There are two roles. A **User** keeps their own wallet — bills, entries, receipts, budgets, notes — and submits receipts. An **Admin** administers the system and reviews the receipts Users submit, but keeps **no wallet of their own**: every personal-finance endpoint refuses an Admin. This separates the person who judges evidence from anyone who submits it (Section 8.2).

6.2 Context Diagram

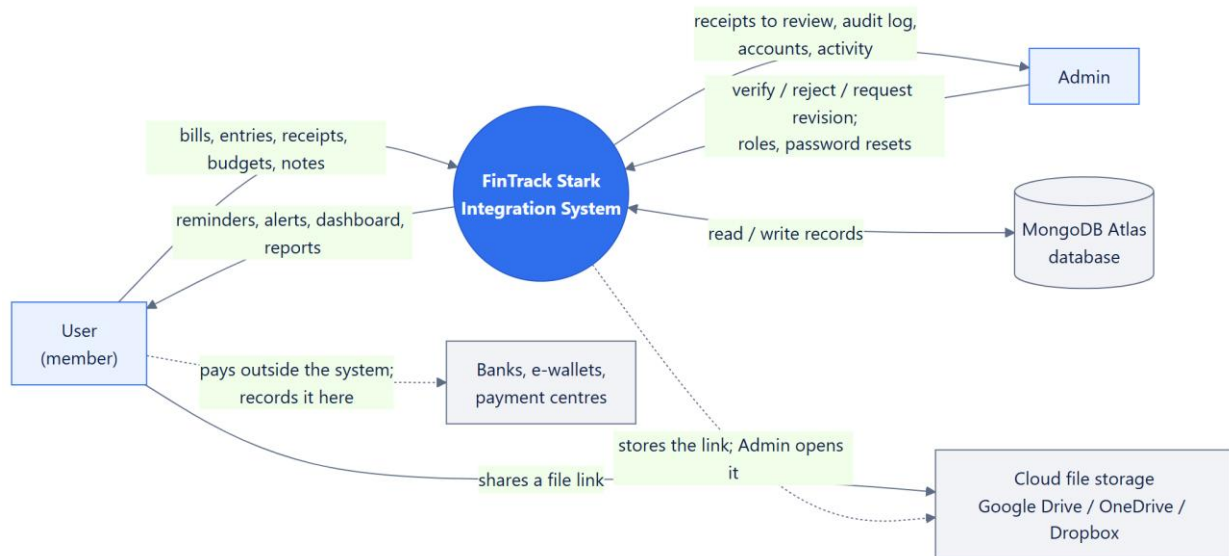


Figure 5. Context Diagram: the system, its users and external systems

6.3 Data-Flow Diagram

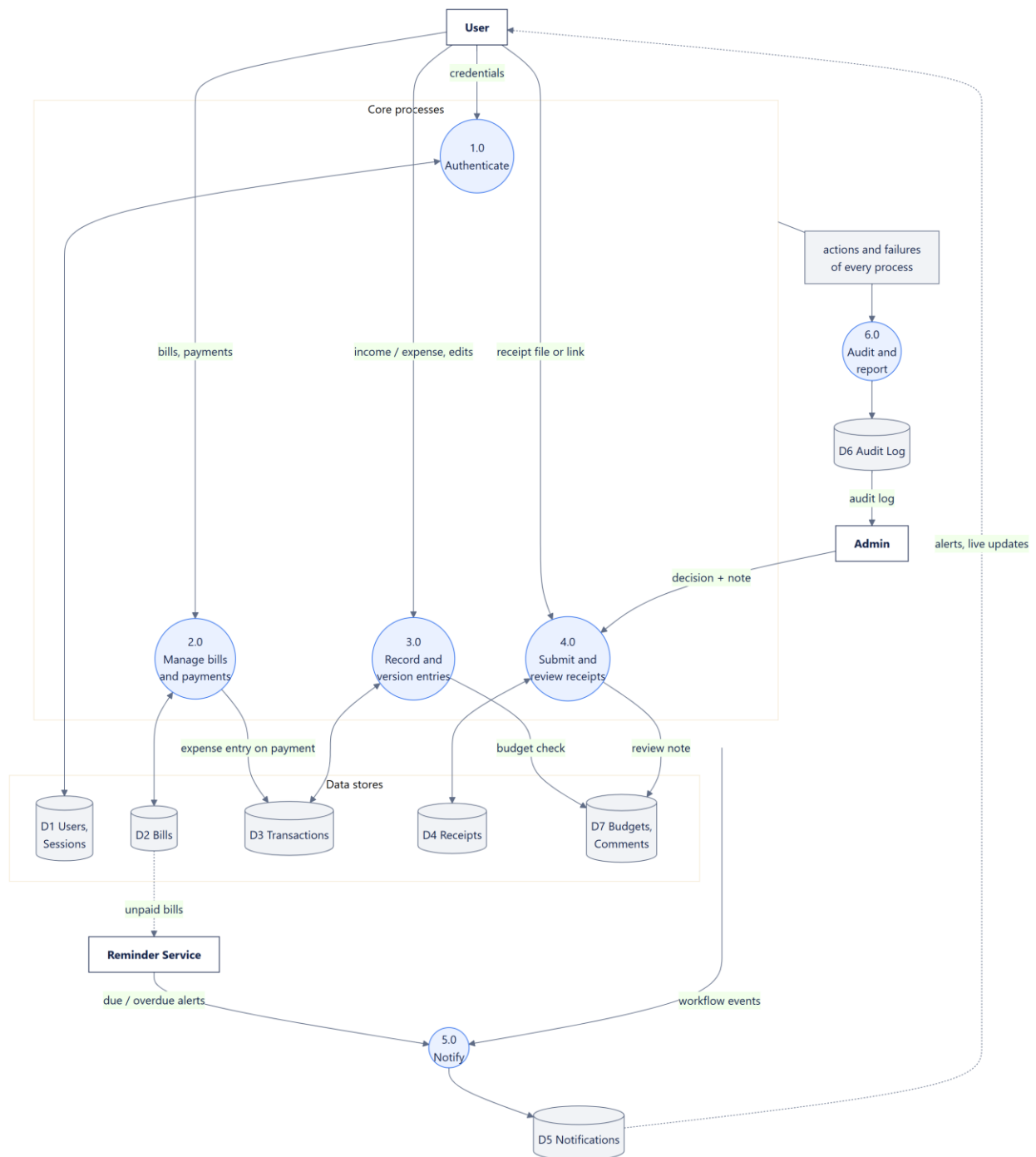


Figure 6. Data-Flow Diagram (Level 1): processes, data stores and flows

Table 16. DFD processes

Process	Description
1.0	Authenticate: checks credentials against D1, creates a session (hashed token) and returns the token.
2.0	Manage bills and payments: stores bills in D2; on payment, writes the expense entry to D3 and raises events.
3.0	Record and version entries: stores entries and versions in D3; checks the month's spending against D7 budgets.

Process	Description
4.0	Submit and review receipts: stores receipt versions in D4; stores the Admin's review note in D7 (comments).
5.0	Notify: turns workflow events and the reminder service's alerts into notifications (D5) and live updates.
6.0	Audit and report: records every action and failure in D6; produces dashboards and reports from D3.

6.4 Sequence Diagram

The sequence diagram below shows one complete interaction — asset (receipt) submission through to final approval — including a revision round. It is exactly the flow exercised by the automated end-to-end test E2E-02.

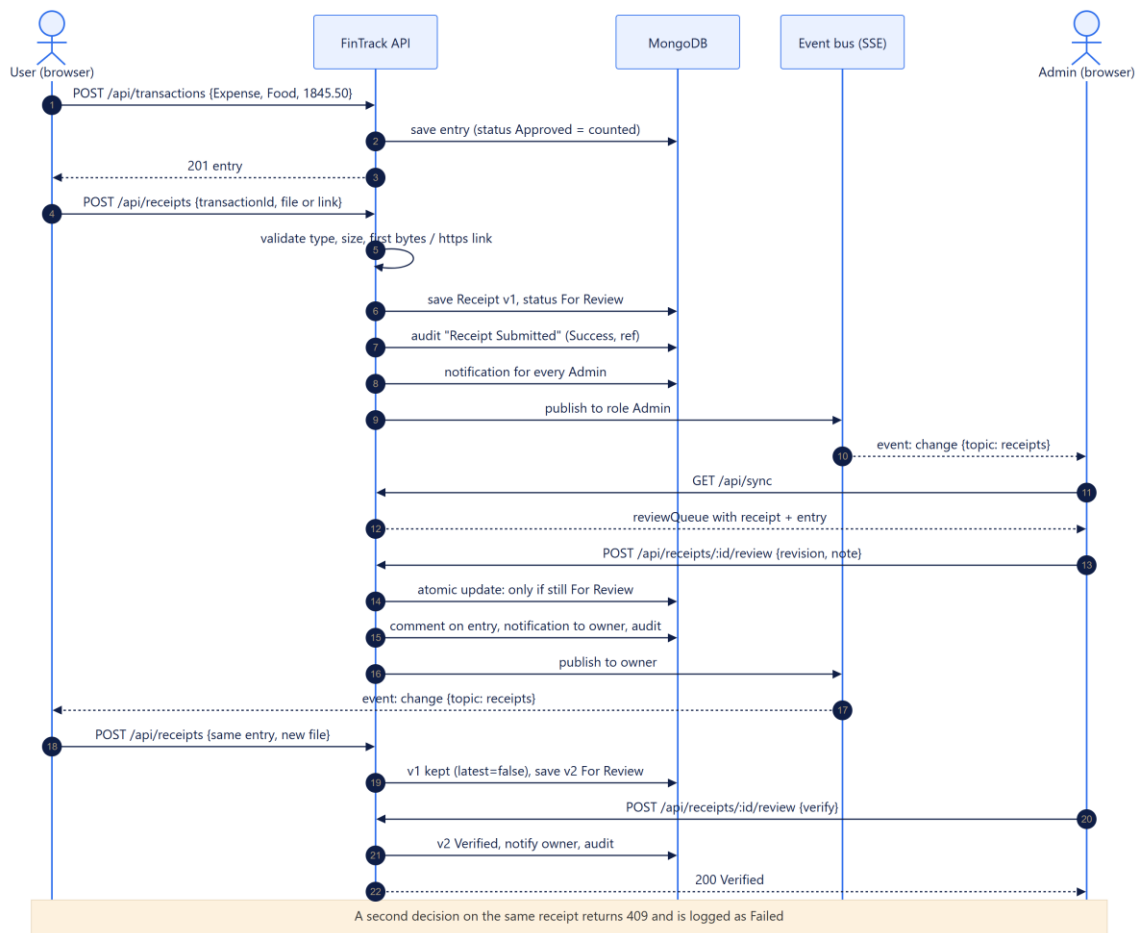


Figure 7. Sequence Diagram: receipt submission, revision request, resubmission (v2) and verification

6.5 Database Design

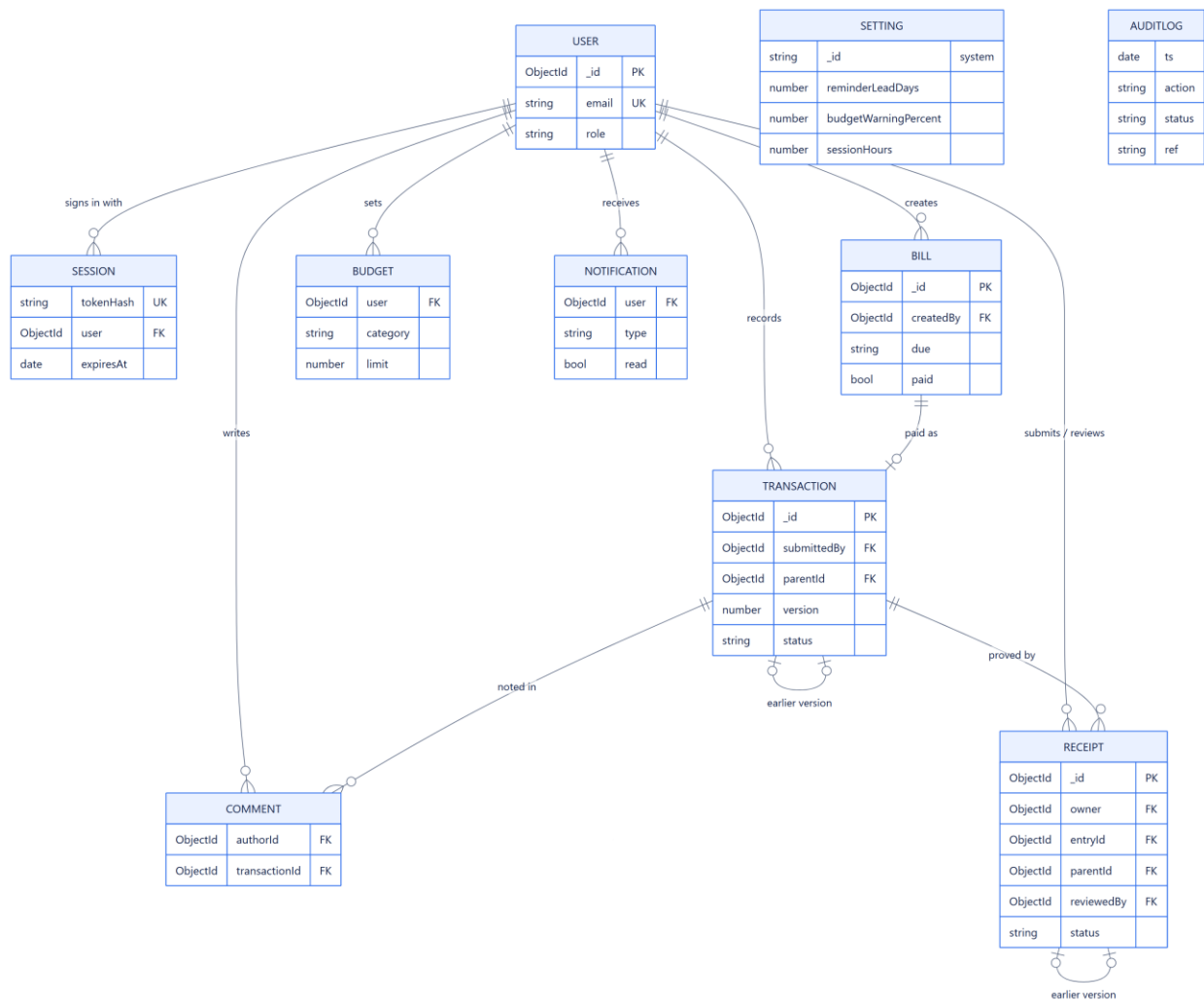


Figure 8. Entity-Relationship Diagram (keys and main fields; every field is listed in the data dictionary below)

MongoDB stores each entity as a collection of documents; relationships are references by Objectid. Version chains (entries and receipts) are self-references through parentId. The following tables are the data dictionary.

Table 17. Collection users

Field	Type	Rules / meaning
_id	Objectid	Primary key
firstName, lastName	String ≤ 60	Required
name	String	Derived full name
email	String ≤ 160	Required, unique, lower-case
passwordHash	String	bcrypt\$N\$r\$p\$salt\$hash (legacy bcrypt still verified)
role	String	Admin User
mustChangePassword	Boolean	True after an Admin reset

Table 18. Collection sessions

Field	Type	Rules / meaning
tokenHash	String	SHA-256 of the bearer token; unique
user	ObjectId → users	Owner
name, email, role	String	Copied at login; refreshed on role/profile change
mustChangePassword	Boolean	Limits the session to changing the password
expiresAt	Date	TTL index; 8 hours

Table 19. Collection bills

Field	Type	Rules / meaning
name	String ≤ 120	Required
category	String	Utilities Housing Internet Credit Subscription Other
amount	Number	0.01 – 1,000,000,000,000
due	String YYYY-MM-DD	Real calendar date
paid, paidOn, paidAmount	Boolean, String, Number	Set atomically by the payment workflow
createdBy	ObjectId → users	Owner
lastRemindedOn	String	Owned by the reminder service
repeat, repeatDay	String, Number	none monthly; day of month 1–31
previousBill	ObjectId → bills	The bill this one followed

Table 20. Collection transactions

Field	Type	Rules / meaning
type	String	Income Expense
category	String ≤ 40	Must match the type's category list
amount	Number	0.01 – 1,000,000,000,000
date	String YYYY-MM-DD	Real date, not in the future
note	String ≤ 300	
status	String	Approved (current, counted) Superseded Deleted
version, parentId	Number, ObjectId	Version chain
autoApproved	Boolean	Created by a bill payment
submittedBy	ObjectId → users	Owner
createdAt	Date	

Table 21. Collection receipts

Field	Type	Rules / meaning
owner, ownerName	ObjectId → users, String	Submitter
entryId	ObjectId → transactions	Follows the entry's current version
version, parentId, latest	Number, ObjectId, Boolean	Receipt version chain
kind	String	file link
fileName, mimeType, size, data	String, String, Number, Binary	≤ 2 MB; JPEG, PNG, WEBP or PDF; data not loaded in lists

Field	Type	Rules / meaning
url, provider	String ≤ 500, String	https only; Google Drive, OneDrive, Dropbox or Web link
status	String	For Review Verified Rejected Needs Revision Replaced Withdrawn
reviewedBy, reviewerName, reviewedAt, reviewNote	ObjectId, String, Date, String ≤ 300	Set once by the decision

Table 22. Collection budgets

Field	Type	Rules / meaning
user	ObjectId → users	Owner
category	String	Expense category; unique per user
limit	Number	1 – 1,000,000,000,000

Table 23. Collection comments

Field	Type	Rules / meaning
transactionId	ObjectId → transactions	Null for a personal note
author, authorId	String, ObjectId → users	Writer
title	String ≤ 120	Optional
text	String ≤ 5000	Required
ts, editedAt	Date	

Table 24. Collection notifications

Field	Type	Rules / meaning
user	ObjectId → users	Addressee
type	String	bill, reminder, overdue, payment, budget, role, receipt, receipt-verified, receipt-revision, receipt-rejected
message	String	
read	Boolean	
ts	Date	

Table 25. Collection settings

Field	Type	Rules / meaning
_id	String	Always "system" (one document)
reminderLeadDays	Number	1–14 days; default 3
budgetWarningPercent	Number	50–95 %; default 80
sessionHours	Number	1–24 hours; default 8; applies to new sign-ins
updatedBy, updatedAt	String, Date	Who changed it last, and when

Table 26. Collection auditlogs

Field	Type	Rules / meaning
ts	Date	Indexed
user	String	Person or service name
action	String	e.g. Payment Recorded, Receipt Verified, Failed: POST bills/:id/pay
status	String	Success Failed
detail	String	What happened; for failures the HTTP status and message
ref	String	Id of the record concerned

6.6 User Interface Design

All twelve screens required by the specification exist in the prototype. The table below maps each one to its screen; the screenshots that follow were captured from the running system with sample data.

Table 27. Required prototype screens

#	Required screen (spec §14)	FinTrack Stark screen	Address
1	Login Page	Log in / Create account	/login, /register
2	Dashboard	Home (User), Admin Console with the receipt review queue (Admin)	/dashboard
3	Project List	Bill Categories	/categories
4	Project Details	Category Detail	/categories/:name
5	Asset or Task Submission Form	Log Income/Expense with receipt	/submit
6	Asset Version or Revision History	Revision History; receipt versions	/revisions, /entries
7	Review and Approval Page	Receipt Review	/review
8	Comment / Feedback Page	Notes / Feedback	/notes
9	Notification Log Page	Notification Log (Inbox)	/notifications
10	Audit Log Page	Audit Log	/audit
11	Reports Page	Reports	/reports
12	User and Role Management Page	User & Role Mgmt	/users

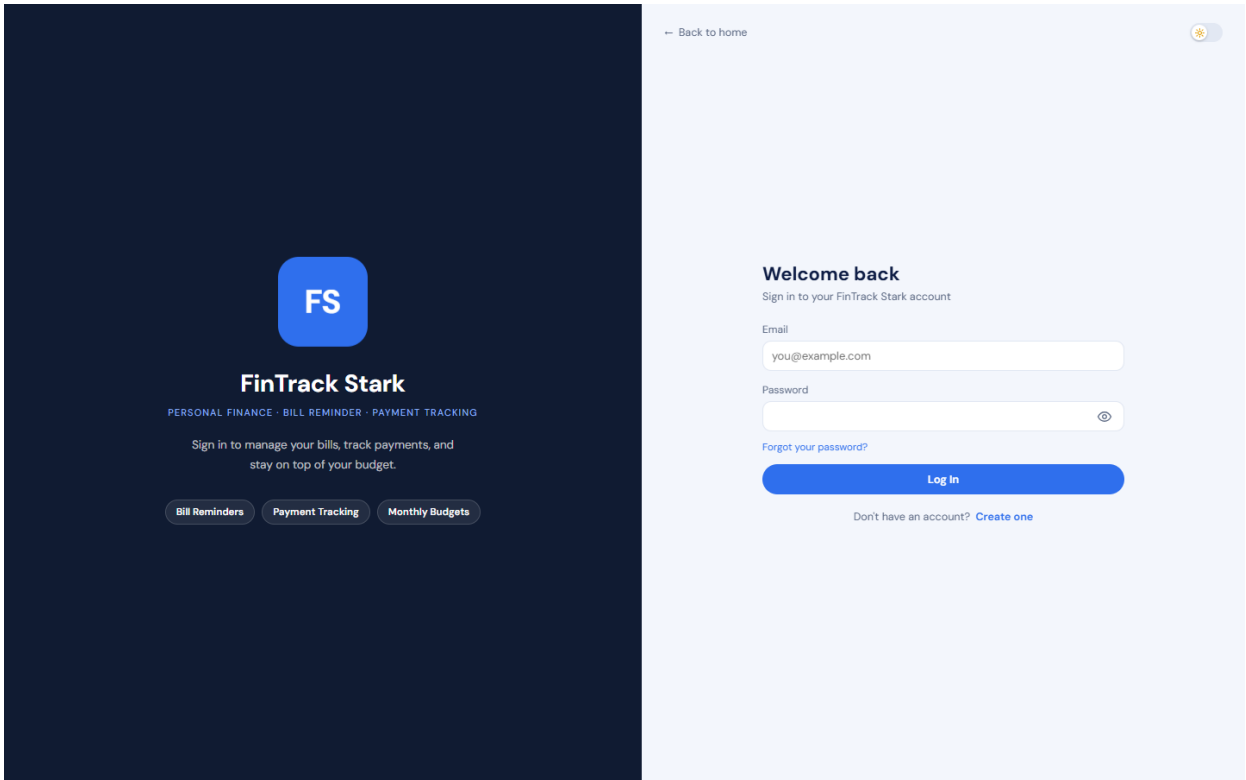


Figure 9. Screen 1 — Login page

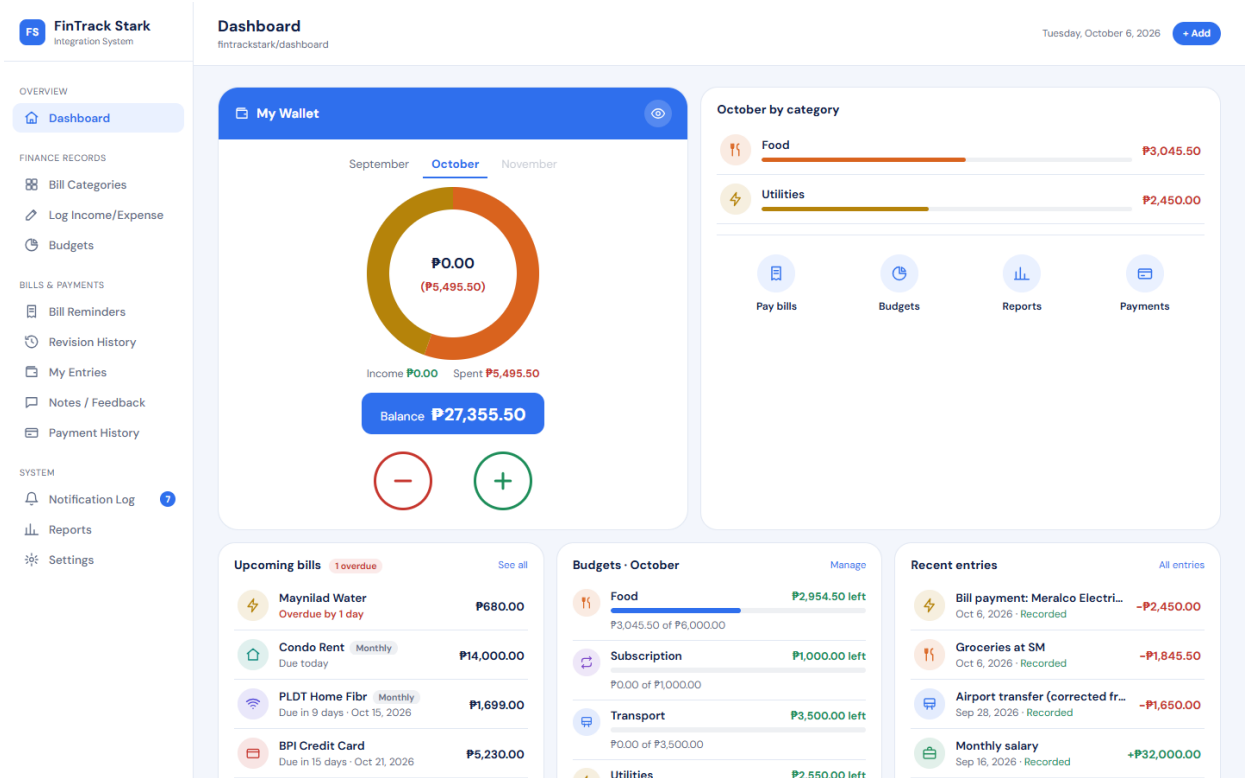


Figure 10. Screen 2 — User dashboard (Home): balance, spending by category, bills and budgets

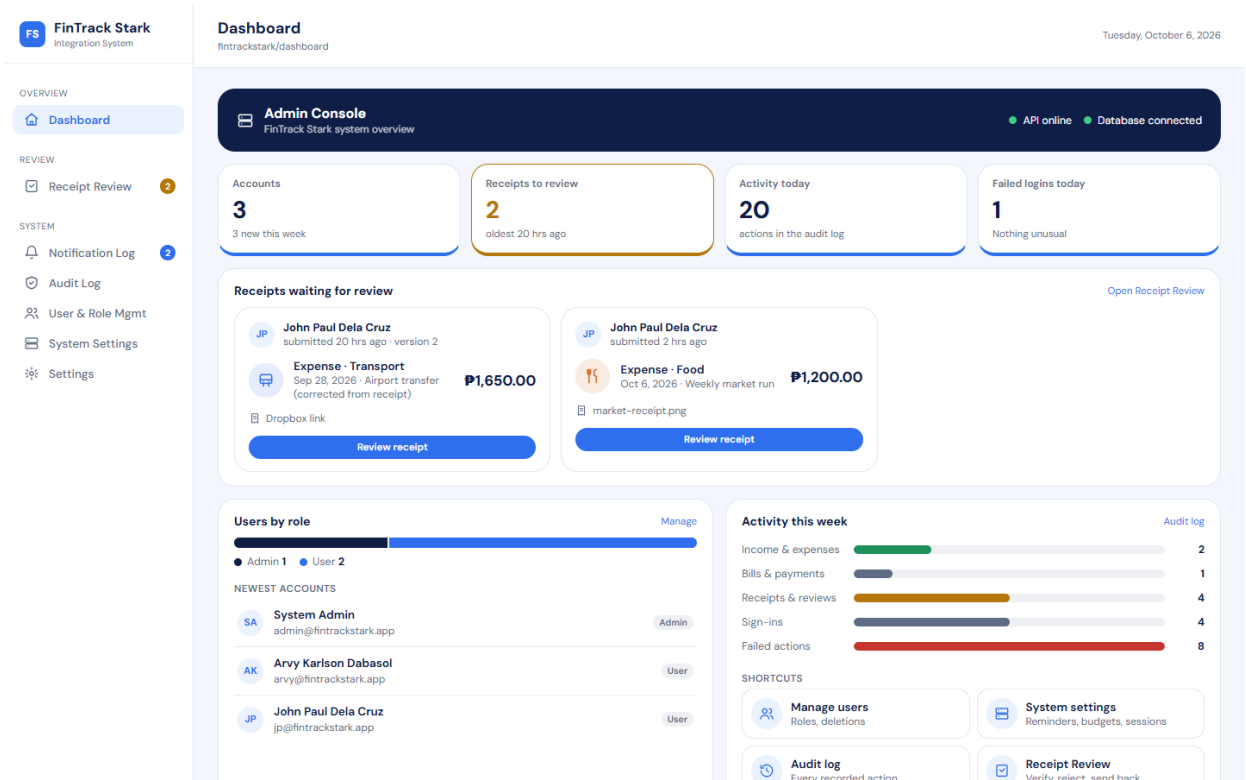


Figure 11. Screen 2 — Admin dashboard (Admin Console): receipts waiting for review, system status, accounts, weekly activity

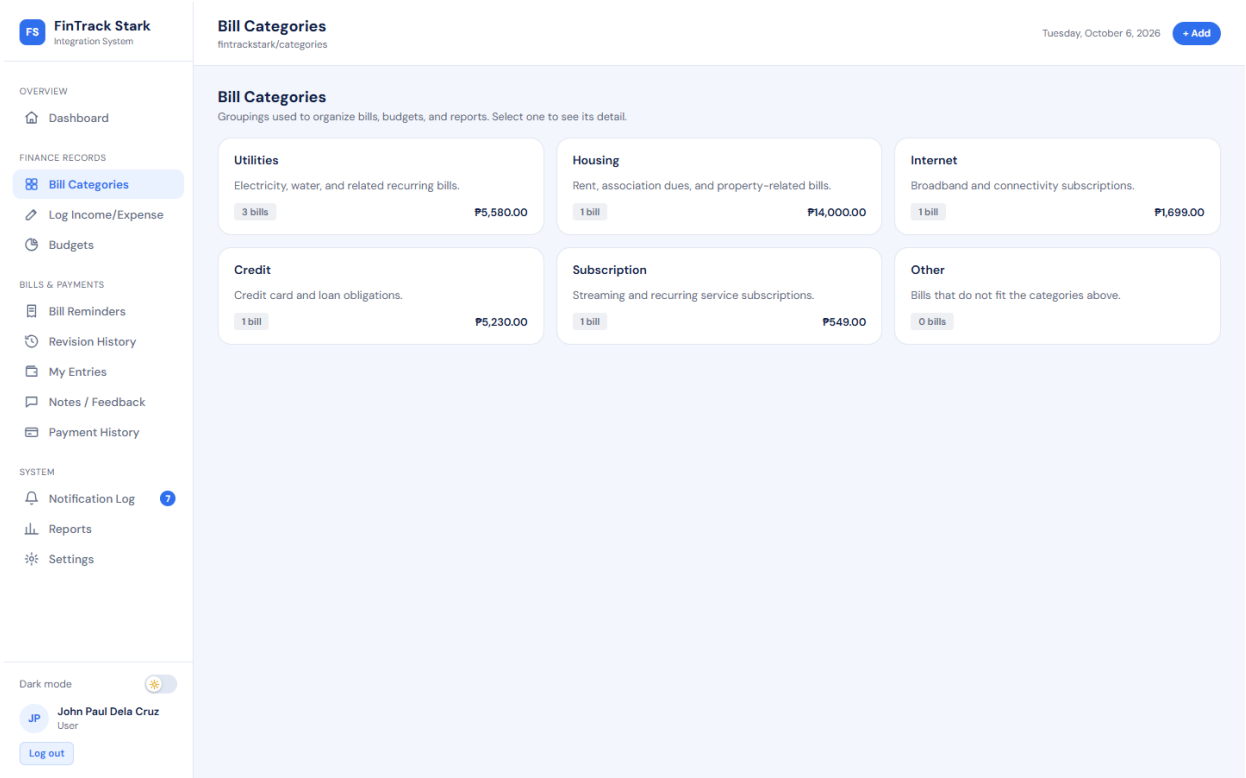


Figure 12. Screen 3 — Project List: Bill Categories

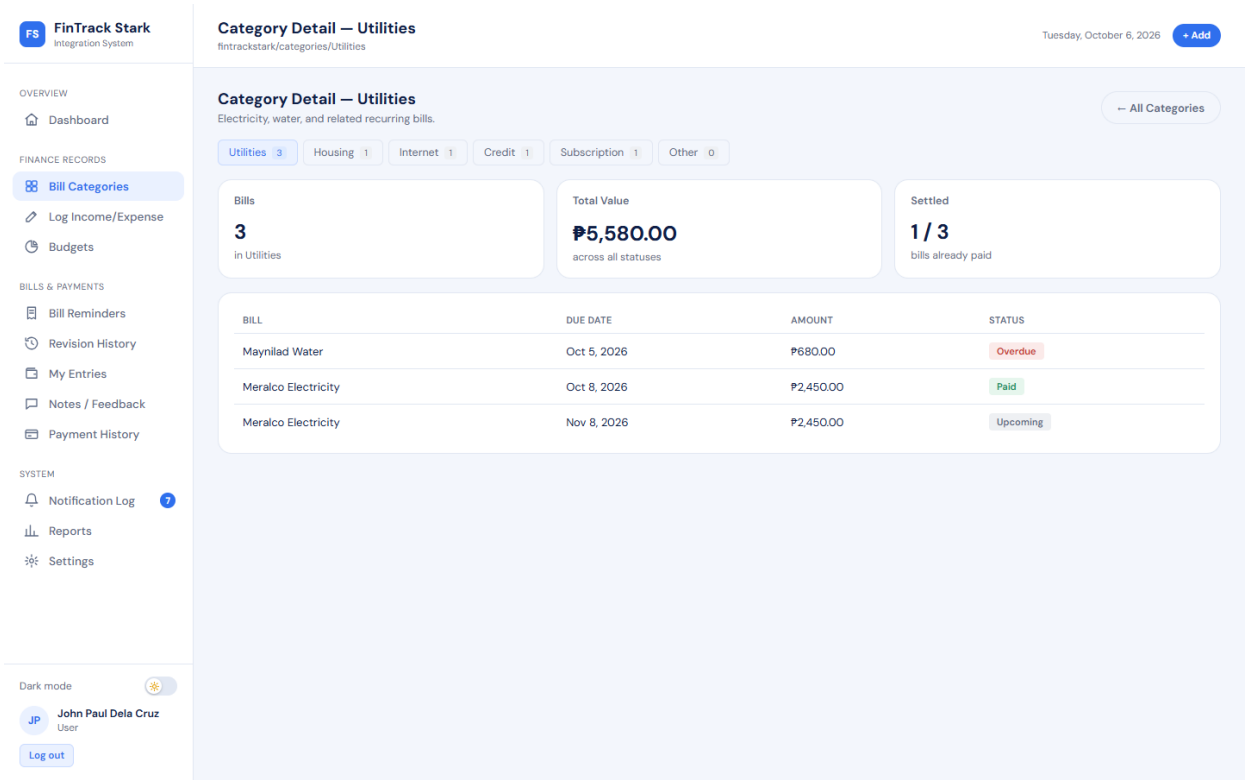


Figure 13. Screen 4 — Project Details: Category Detail (Utilities)

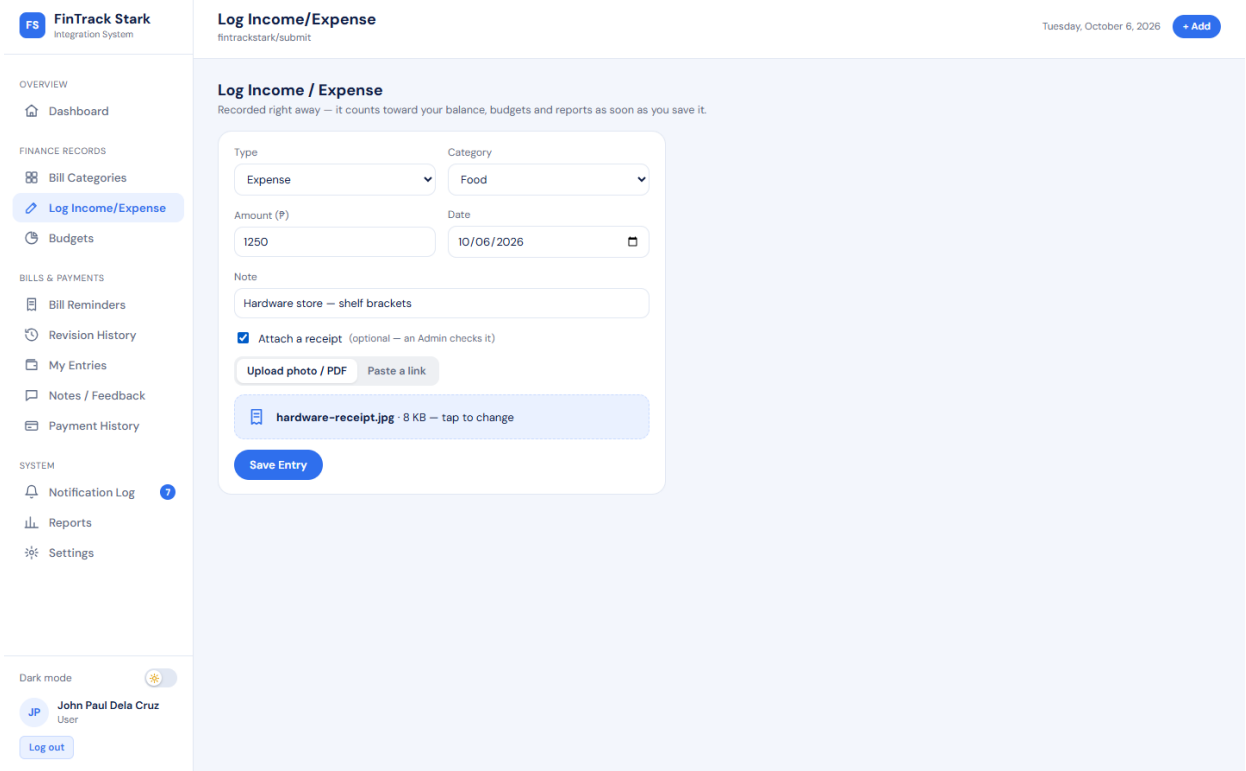


Figure 14. Screen 5 — Submission form: Log Income/Expense with a receipt photo attached

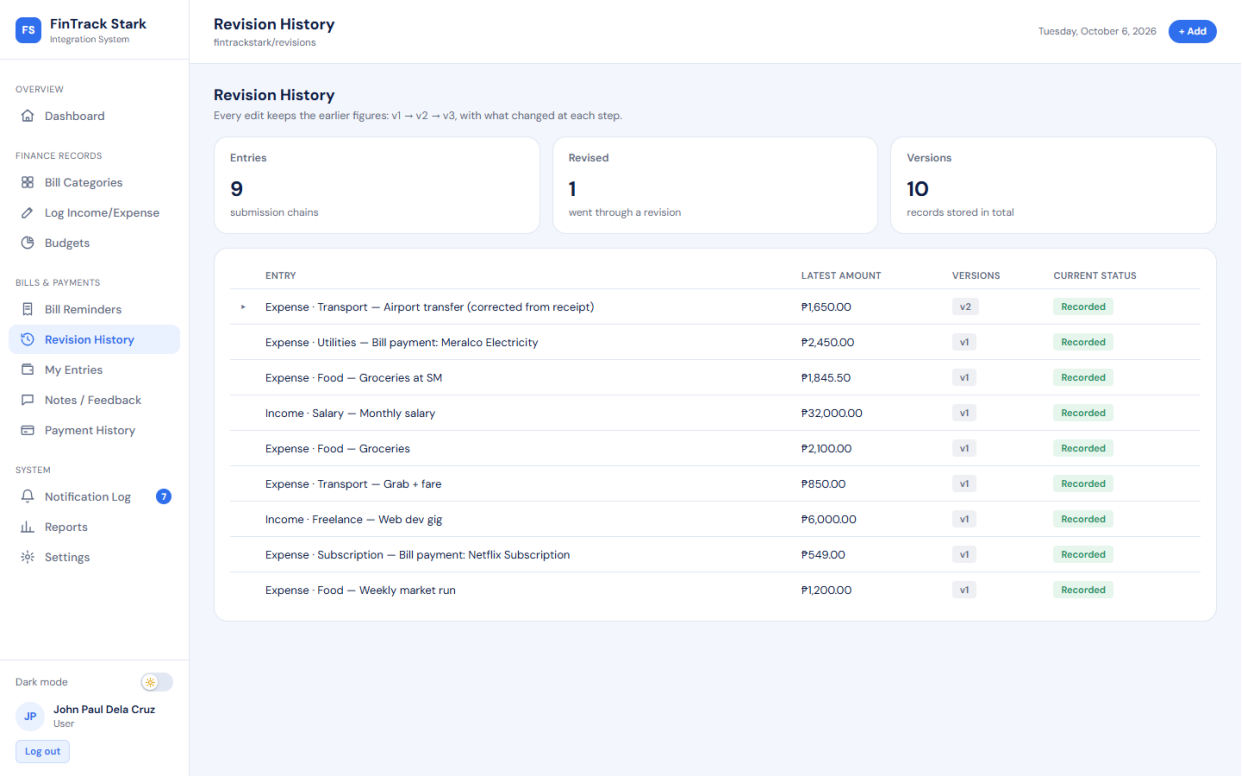


Figure 15. Screen 6 — Revision History: entry versions (v1 → v2)

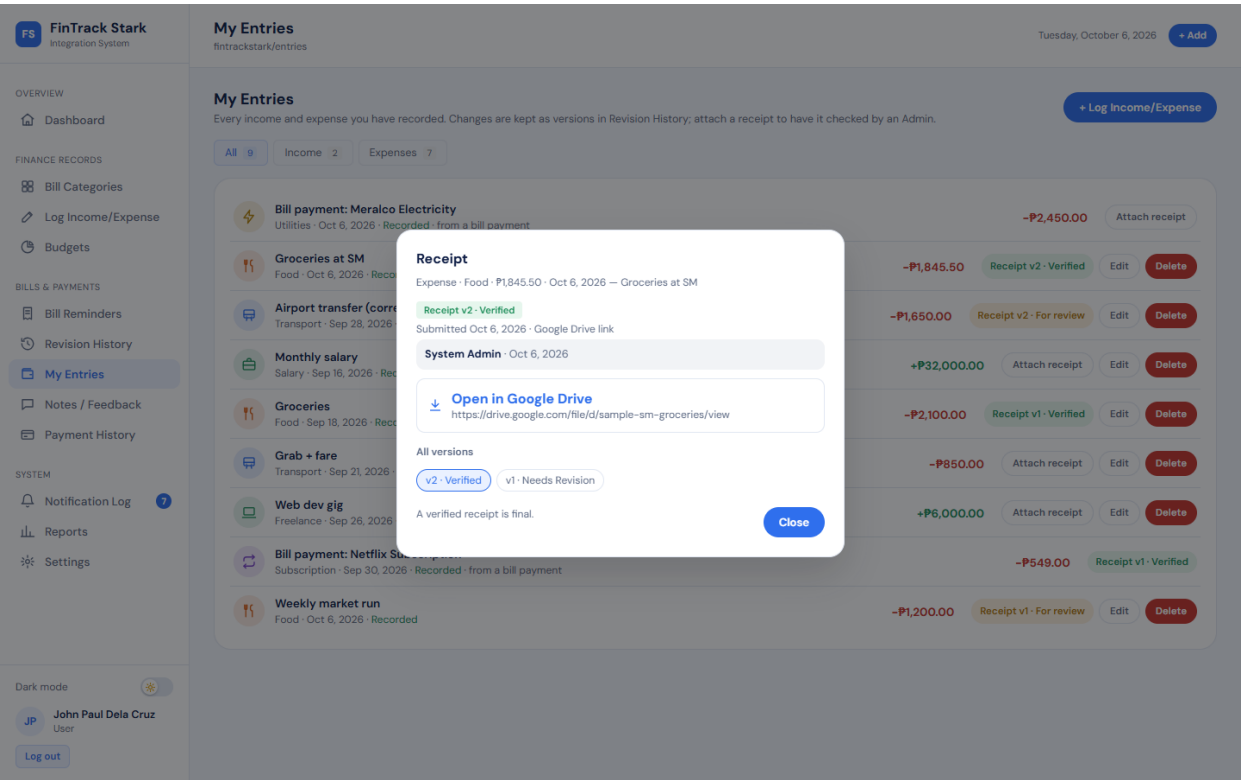


Figure 16. Screen 6 — Receipt versions: v1 Needs Revision, v2 Verified (Google Drive link)

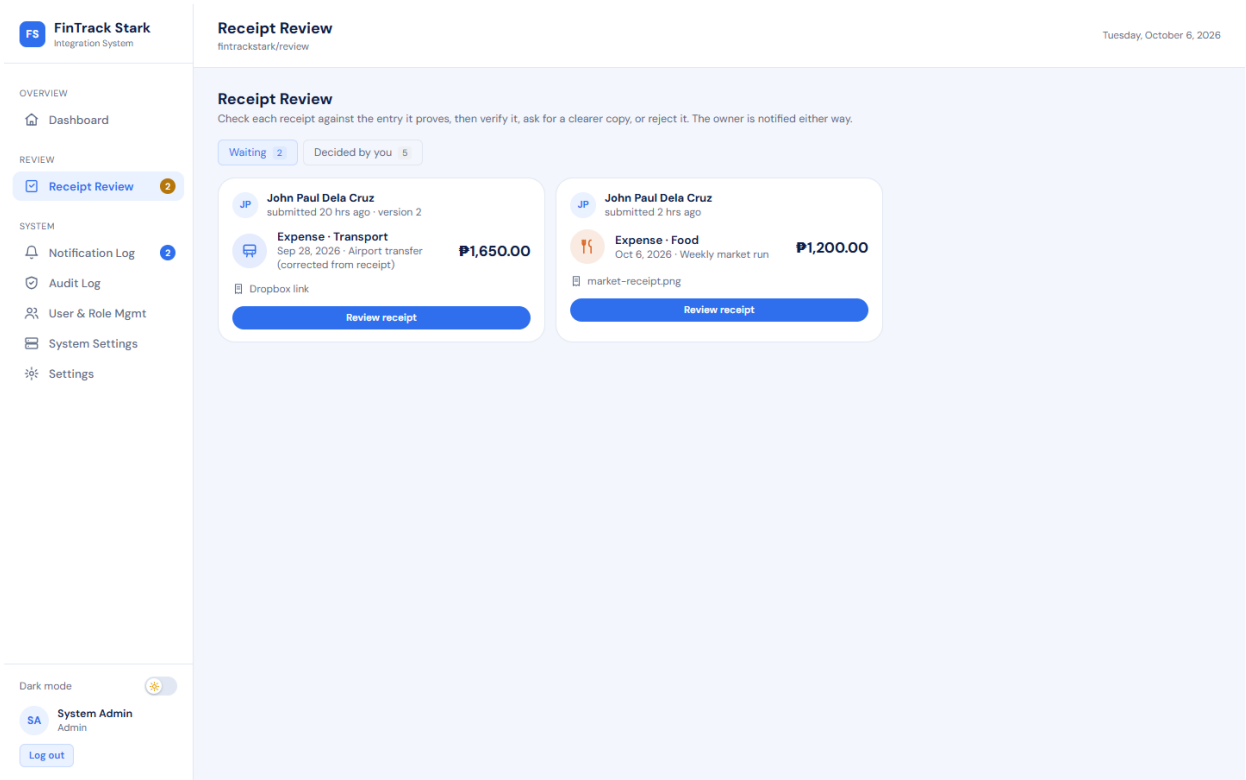


Figure 17. Screen 7 — Review and Approval page: Receipt Review queue

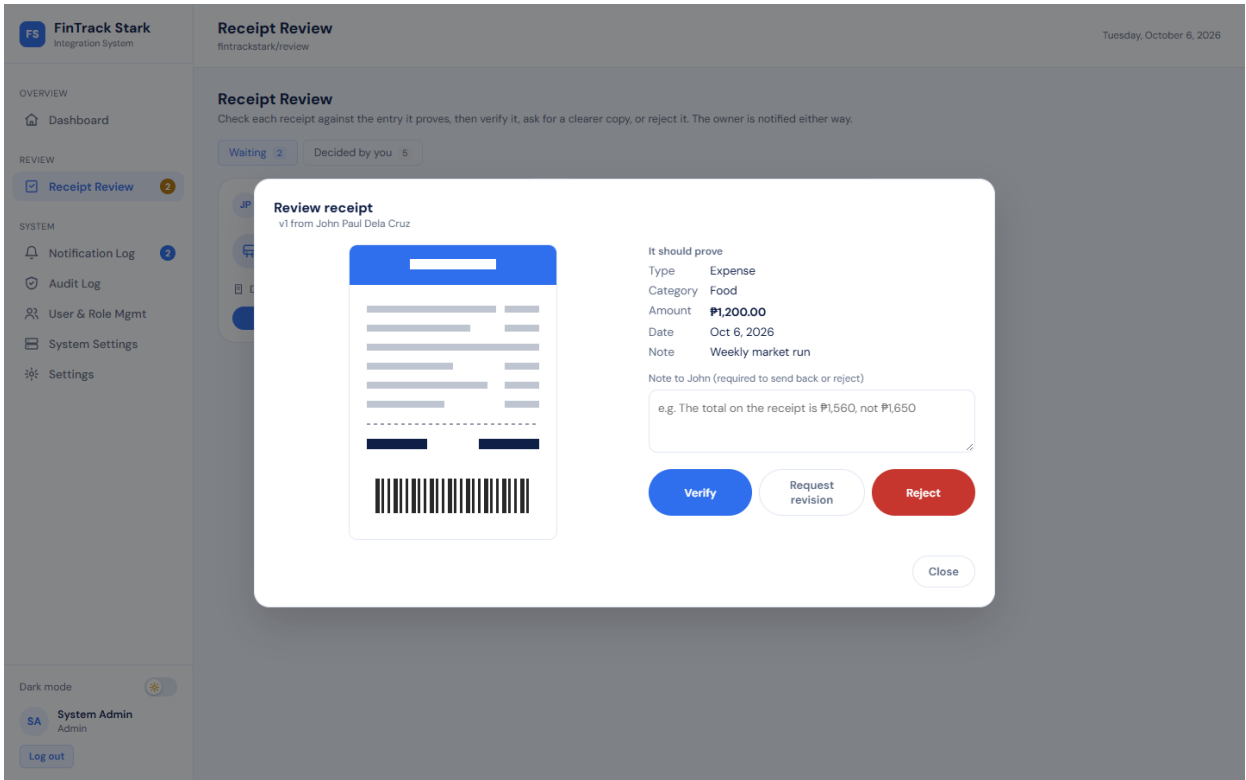


Figure 18. Screen 7 — Reviewing a receipt against its entry: Verify, Request revision, Reject

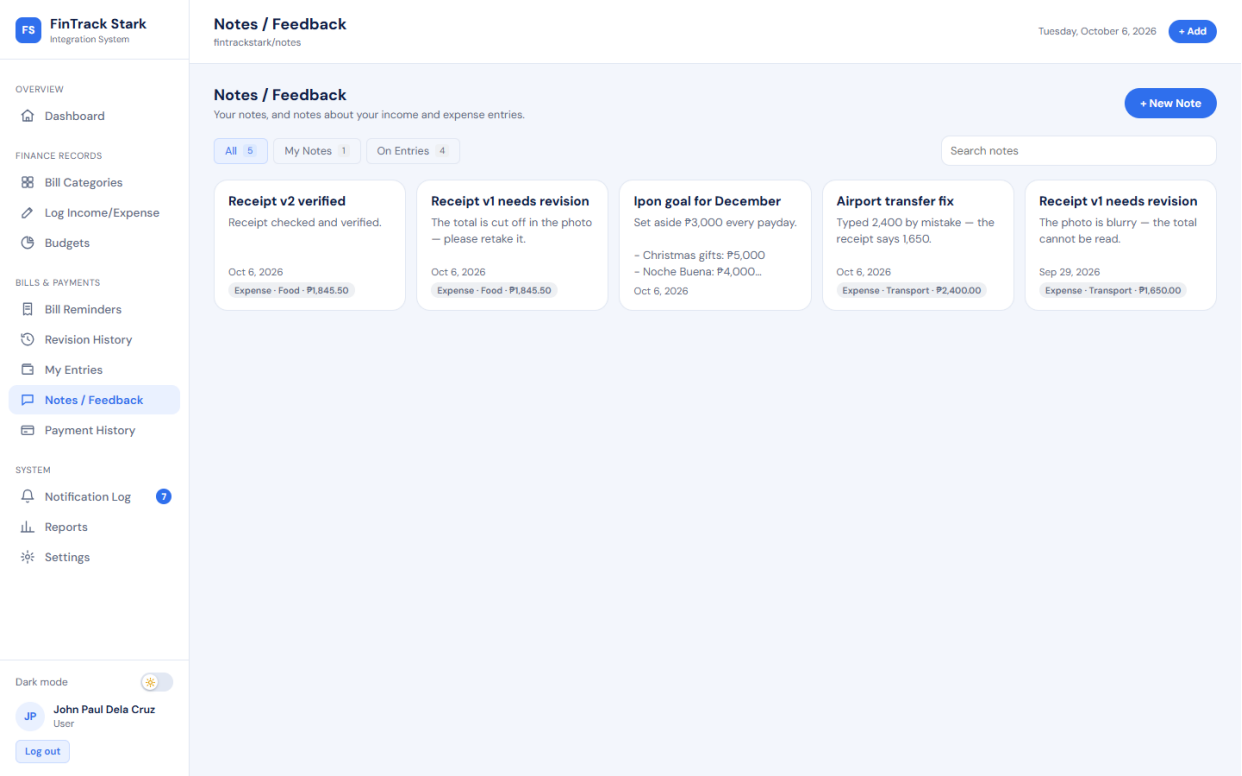


Figure 19. Screen 8 — Comment / Feedback page: Notes / Feedback

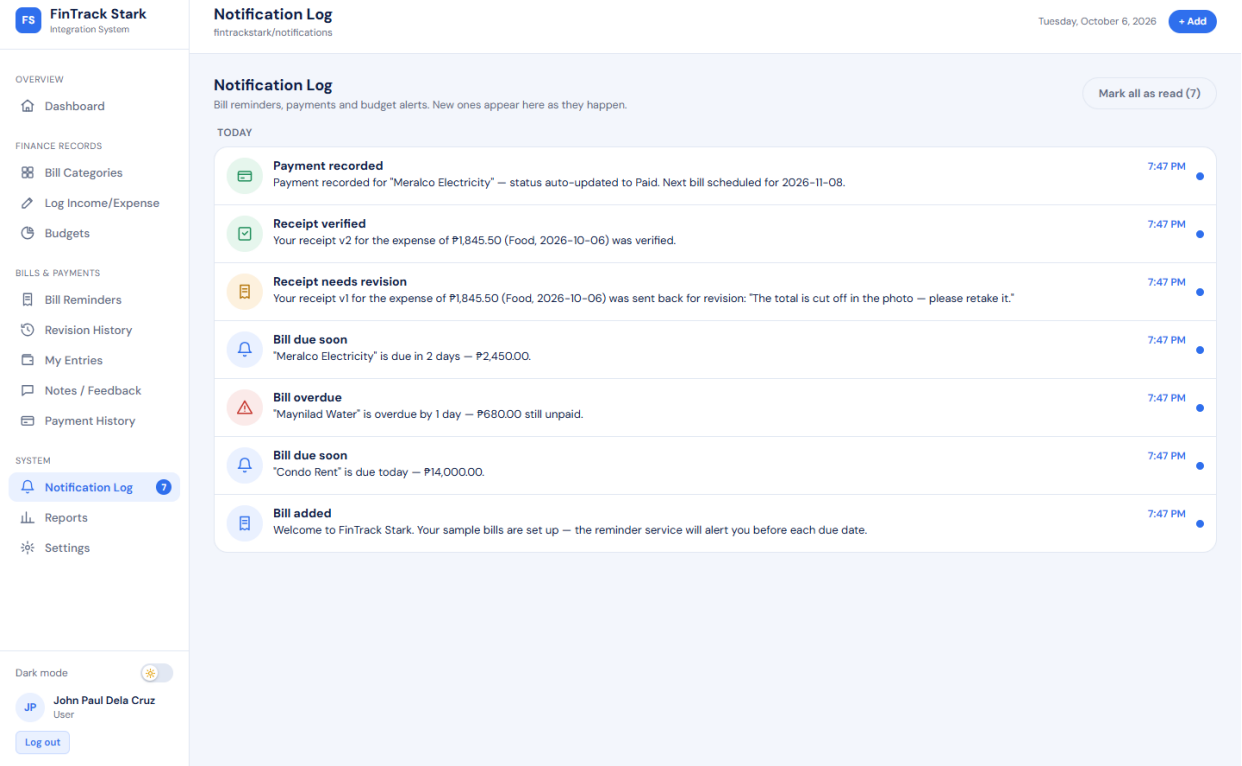


Figure 20. Screen 9 — Notification Log

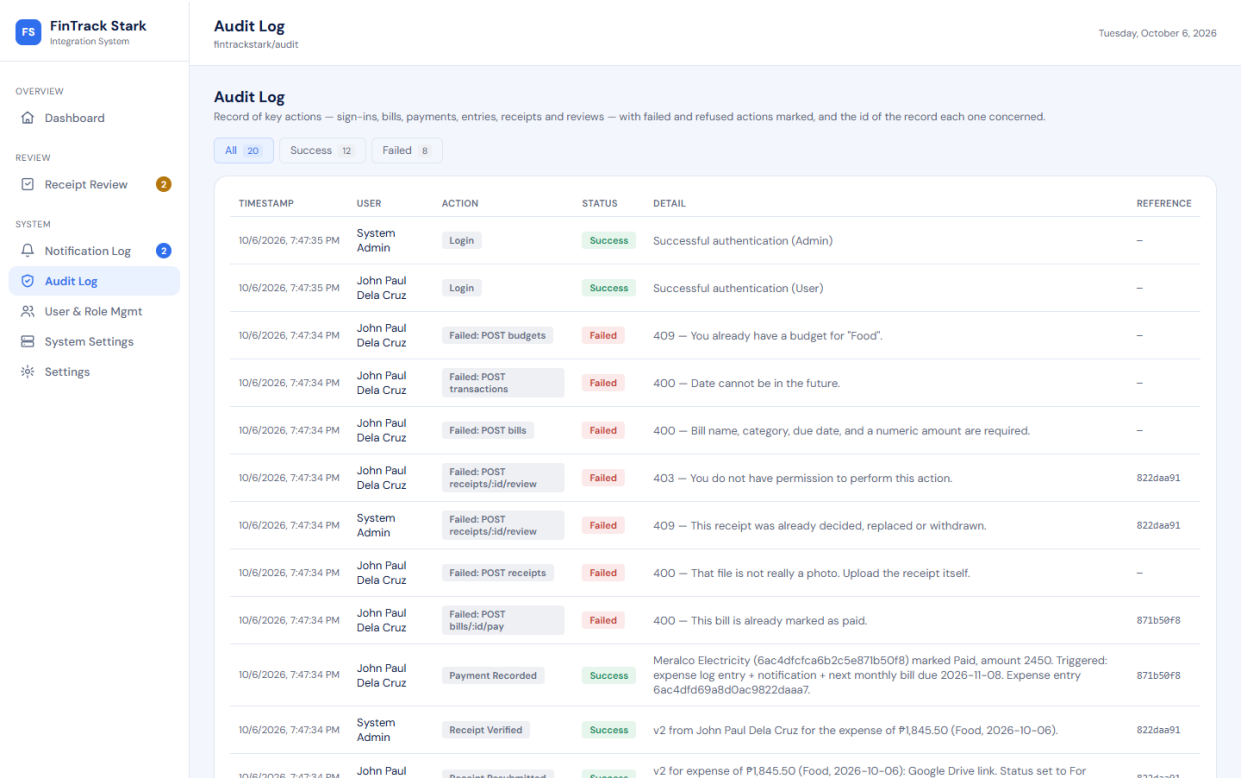


Figure 21. Screen 10 — Audit Log with status and record reference

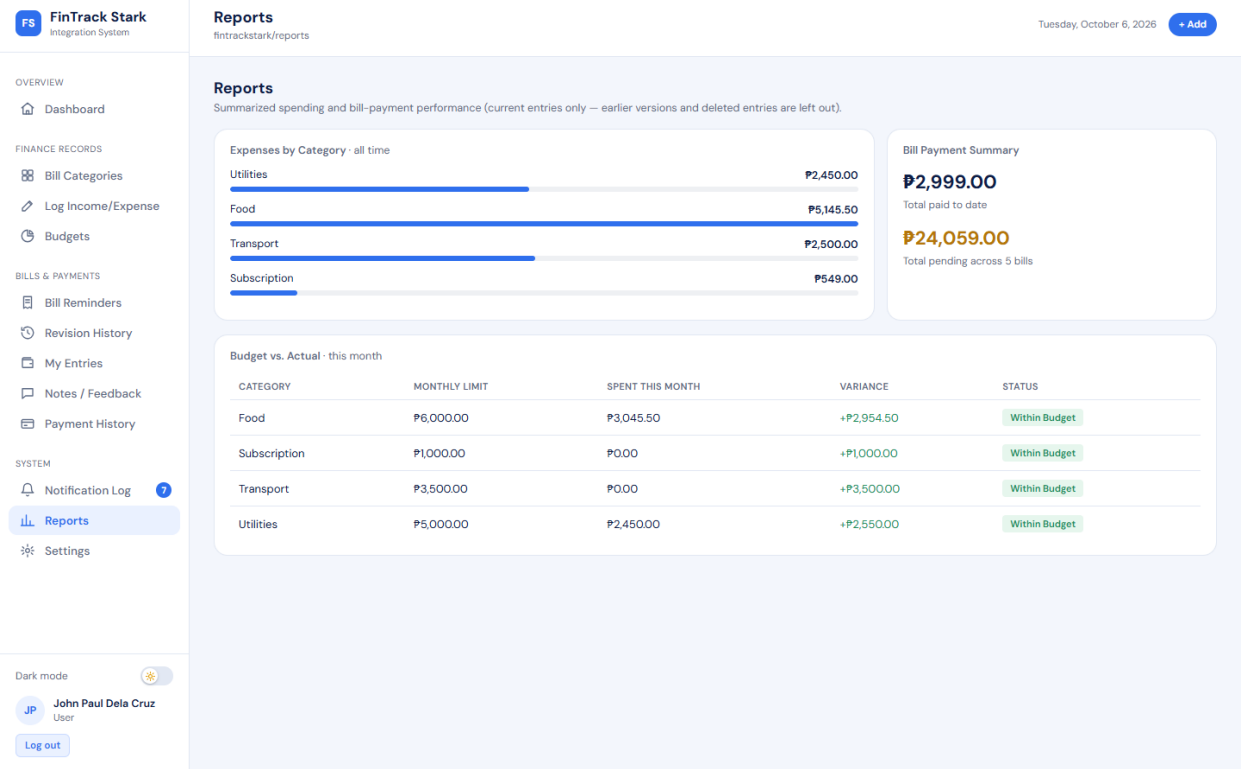


Figure 22. Screen 11 — Reports

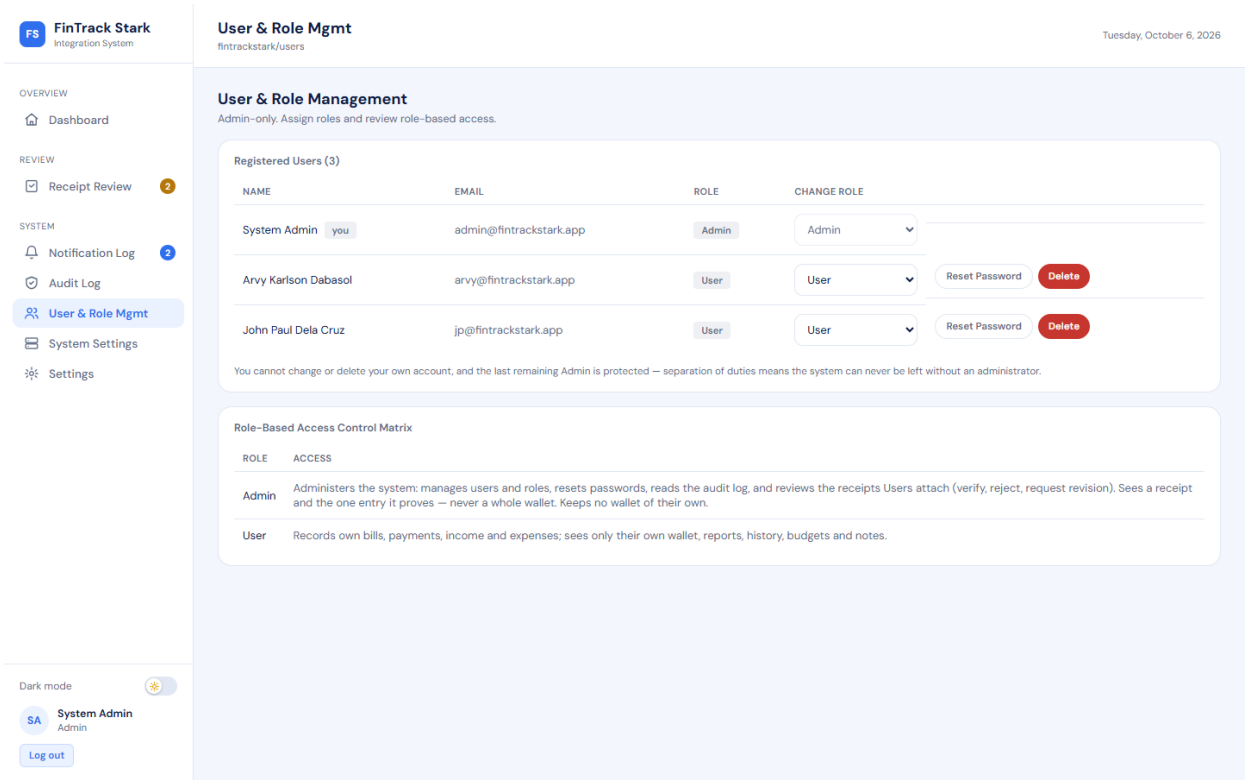


Figure 23. Screen 12 — User and Role Management

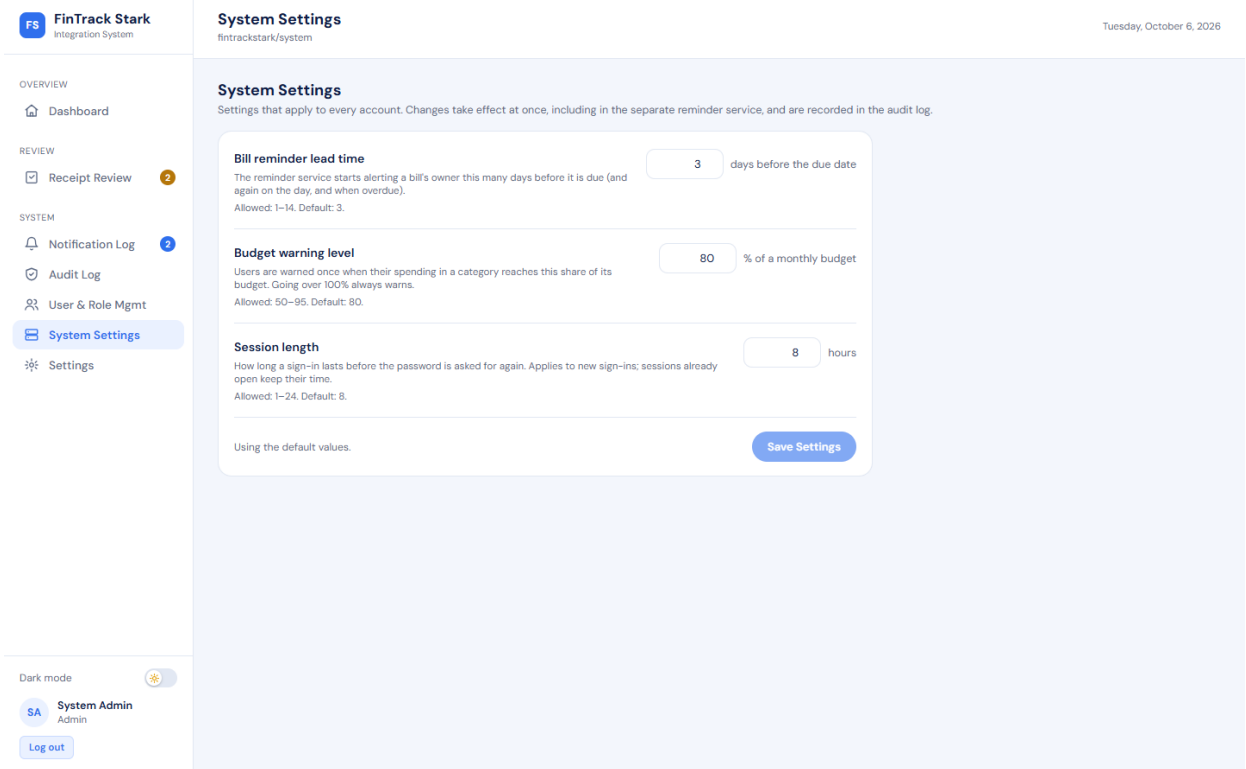


Figure 24. Additional Admin screen — System Settings (reminder lead time, budget warning level, session length)

7. Integration Design and Implementation

7.1 Integration Scenario

FinTrack Stark integrates six connections. Together they cover five of the specification's integration options: **API integration**, **workflow automation**, **webhook simulation**, **messaging simulation** and **external storage integration simulation**.

Table 28. Integrations implemented

#	Integration	What is connected	Spec §6 option	Pattern
I1	Client ↔ REST API	React client and Express API: 42 endpoints, JSON, Bearer token	API Integration	Point-to-point
I2	Bill payment cascade	Bills → Transactions → Notifications → Audit → next Bill → Budgets	Workflow Automation, Webhook Simulation	Internal event chain
I3	Receipt submission and review	Entries → Receipts → Admin review queue → decision → Comments, Notifications, Audit	Workflow Automation, Webhook Simulation	Internal event chain
I4	Live event messaging	API event bus → every open browser of the addressed user or role	Messaging Simulation	Event-driven (SSE)
I5	Reminder service	Separate scheduled process ↔ bills and notifications	Workflow Automation	Shared database
I6	Cloud file links	Receipt ↔ Google Drive, OneDrive, Dropbox	External Storage Integration Simulation	Reference by URL

7.2 API / Workflow / Webhook / Messaging Components

I2 — Bill payment cascade (workflow automation and webhook simulation)

One action — **Mark Paid** — triggers the whole chain shown below. The bill is claimed with an atomic conditional update (`paid: false → true`), so a double click or a second tab cannot pay it twice; if the expense entry cannot be written, the bill is put back to unpaid so a bill never reads *Paid* without the payment that paid it.

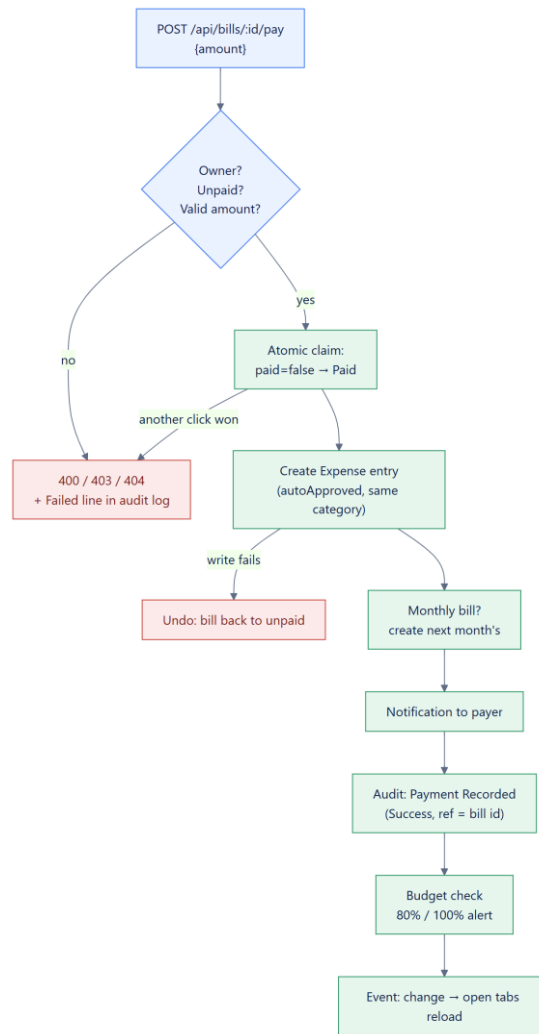


Figure 25. Bill payment cascade (I2)

I3 — Receipt submission and review (workflow automation)

Submitting a receipt automatically sets it to **For Review** and notifies every Admin; each decision automatically notifies the owner, adds the decision as a note on the entry and writes to the audit log. The life cycle of a receipt is:

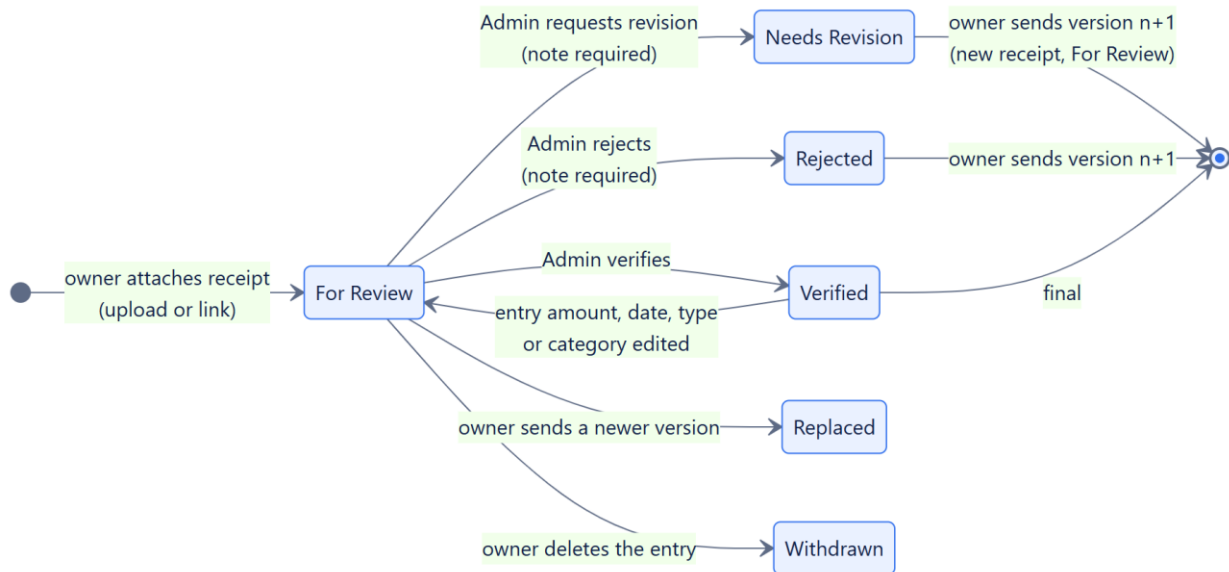


Figure 26. Receipt status life cycle (I3)

- A decision is accepted only while the receipt is still *For Review* and still the latest version (atomic update); a second decision returns **409** and is logged as Failed.
- Nobody may review their own receipt (separation of duties); a note is required to reject or to request a revision.
- If the entry's type, category, amount or date is edited after verification, the receipt goes back to *For Review*, because it no longer proves the new figures. Deleting the entry withdraws a receipt still waiting.

I4 — Live event messaging (messaging simulation)

Every change publishes a small event on the API's event bus addressed to a user or a role. Each open browser tab keeps a Server-Sent Events connection (`GET /api/events`) and, on an event, reloads its data through `GET /api/sync`. Events carry only a topic — never data — so the authorisation rules of the normal API still decide what each person sees. The stream below was captured from a User's open connection while the workflow in Section 7.4 ran:

Captured event stream (`GET /api/events`)

```

retry: 3000
event: ready
data: {}

event: change
data: {"topic":"transactions"}

event: change
data: {"topic":"receipts"}

event: change
data: {"topic":"notifications"}

event: change
data: {"topic":"receipts"}

event: change
data: {"topic":"comments"}

```

```
event: change
data: {"topic":"receipts"}
```

I5 — Reminder service (shared-database integration)

The reminder service is a separate Node.js process (services/reminder, scheduled with node-cron at 08:00 daily); the API runs the same sweep on start-up and on the first request of each Philippine day so that reminders still happen on hosting where only one process runs. Both find unpaid bills due within three days or overdue, claim each bill for the day (lastRemindedOn), insert one notification per bill for its owner, and record the sweep in the audit log. The worker never calls the API — it even reads the reminder lead time the Admin sets in System Settings from the shared database.

Reminder service run, captured from the terminal (second run shows no duplicates)

```
> npm run reminder:once
[db] connected to mongodb://127.0.0.1:27017/fintrack_stark_docs
[reminder] 2026-10-06: raised 2 alert(s) across 1 account(s).

> npm run reminder:once      (run again the same day)
[db] connected to mongodb://127.0.0.1:27017/fintrack_stark_docs
[reminder] 2026-10-06: 2 bill(s) in range, none need a new alert.
```

I1 — Main API endpoints

Table 29. Main REST endpoints used by the integrations (full list in Appendix A)

Method	Path	Who	Purpose
POST	/api/auth/login	Anyone	Sign in; returns a session token
GET	/api/sync	Signed in	Everything the screens show, in one response
GET	/api/events	Signed in	Live event stream (SSE)
POST	/api/transactions	Signed in	Record income or expense
PUT	/api/transactions/:id	Owner	Edit → new version
POST	/api/bills/:id/pay	Owner	Payment cascade (I2)
POST	/api/receipts	Owner of the entry	Submit receipt v1 or a new version (I3)
GET	/api/receipts/:id/file	Owner, Admin	Open an uploaded receipt
GET	/api/receipts/review	Admin	Review queue
POST	/api/receipts/:id/review	Admin	Verify / reject / request revision
GET	/api/audit-log?status=Failed	Admin	Integration log (failed actions)
PUT	/api/settings	Admin	Change system settings (audited)

7.3 Data Mapping Table

Table 30. Mapping: Bill → Expense entry (payment cascade)

Source (bill / request)	Target (transaction)	Rule
—	type	Always "Expense"
bill.category	category	Copied (bill categories are a subset of expense categories, so the budget applies)
request.amount (default bill.amount)	amount	Validated 0.01 – 1e12; also stored as bill.paidAmount

Source (bill / request)	Target (transaction)	Rule
today (Asia/Manila)	date; bill.paidOn	Philippine calendar date, not UTC
bill.name	note	"Bill payment: <name>"
—	status / autoApproved	"Approved" (counted) / true — cannot be edited separately
session user	submittedBy	The payer (always the bill's owner)
bill.repeat = monthly	new bill.due	Same day next month (clamped to month length); previousBill = paid bill

Table 31. Mapping: Receipt submission request → Receipt document

Request field	Receipt field	Rule
transactionId	entryId	Must be the caller's current entry
kind	kind	"file" or "link"
data (base64)	data (binary), size	Decoded; ≤ 2 MB; first bytes must match mimeType
mimeType	mimeType	image/jpeg, image/png, image/webp or application/pdf
fileName	fileName	Path removed; unsafe characters replaced; ≤ 120
url	url, provider	https only; provider derived from the host name
session user	owner, ownerName	
previous latest receipt	version, parentId	version = previous + 1; previous.latest = false
—	status	"For Review" (automatic)

Table 32. Mapping: Review decision → records written

Decision field	Target	Value
action	receipt.status	verify → Verified; revision → Needs Revision; reject → Rejected
note	receipt.reviewNote; comment.text	Required for revision and reject; ≤ 300
session user	receipt.reviewedBy, reviewerName; comment.author	
receipt.version, decision	comment.title	"Receipt v<n> <status>"
receipt.owner	notification.user	type receipt-verified / receipt-revision / receipt-rejected
all of the above	auditlog	action "Receipt Verified" etc., status Success, ref = receipt id

Table 33. Mapping: Bill → reminder notification (reminder service)

Source (bill)	Target (notification)	Rule
createdBy	user	Addressed to the bill's owner only
due vs. today	type	overdue (past due) or reminder (due within 3 days or today)
name, amount, due	message	e.g. "Meralco Electricity" is due in 2 days — ₱2,450.00.
today	bill.lastRemindedOn	Claim: at most one alert per bill per day

7.4 Sample Payload or Data Format

The following requests and responses were captured from the running system (local server with sample data) on 10/6/2026, 19:47:33 (Philippine time). Response times are the server's actual times.

1. Sign in — request

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{
  "email": "jp@fintrackstark.app",
  "password": "....."
}
```

1. Sign in — response

```
HTTP/1.1 200 (134 ms)
Content-Type: application/json

{
  "token": "fb1bdb31...(64 hex characters, shown once)",
  "user": {
    "id": "6ac4dfcfca6b2c5e871b50ed",
    "name": "John Paul Dela Cruz",
    "email": "jp@fintrackstark.app",
    "role": "User",
    "mustChangePassword": false
  }
}
```

2. Record an expense — request and response

```
POST /api/transactions HTTP/1.1
Authorization: Bearer <session token>
Content-Type: application/json
```

```
{
  "type": "Expense",
  "category": "Food",
  "amount": 1845.5,
  "date": "2026-10-06",
  "note": "Groceries at SM"
}
```

```
HTTP/1.1 201 (10 ms)
Content-Type: application/json
```

```
{
  "type": "Expense",
  "category": "Food",
  "amount": 1845.5,
  "date": "2026-10-06",
  "note": "Groceries at SM",
  "status": "Approved",
  "version": 1,
  "parentId": null,
  "autoApproved": false,
  "submittedBy": "6ac4dfcfca6b2c5e871b50ed",
  "reviewedBy": null,
  "reviewComment": "",
  "_id": "6ac4dfd59a8d0ac9822daa6f",
  "createdAt": "2026-10-06T11:47:33.628Z",
  "__v": 0
}
```

3. Submit a receipt (asset submission) — request

```
POST /api/receipts HTTP/1.1
Authorization: Bearer <session token>
Content-Type: application/json

{
  "transactionId": "6ac4dfd59a8d0ac9822daa6f",
  "kind": "file",
  "fileName": "sm-groceries.jpg",
  "mimeType": "image/png",
  "data": "iVBORw0KGGoAAAANSUhEUgAAAPAAAFUCAIAAACV...(base64, 2160 chars)"
}
```

3. Submit a receipt — response (status set to For Review automatically)

```
HTTP/1.1 201 (12 ms)
Content-Type: application/json

{
  "_id": "6ac4dfd59a8d0ac9822daa78",
  "owner": "6ac4dfcfca6b2c5e871b50ed",
  "ownerName": "John Paul Dela Cruz",
  "entryId": "6ac4dfd59a8d0ac9822daa6f",
  "version": 1,
  "parentId": null,
  "latest": true,
  "kind": "file",
  "fileName": "sm-groceries.jpg",
  "mimeType": "image/png",
  "size": 1618,
  "url": "",
  "provider": "",
  "status": "For Review",
  "reviewerName": "",
  "reviewedAt": null,
  "reviewNote": "",
  "submittedAt": "2026-10-06T11:47:33.644Z"
}
```

4. Admin requests a revision — request and response

```
POST /api/receipts/6ac4dfd59a8d0ac9822daa78/review HTTP/1.1
Authorization: Bearer <session token>
Content-Type: application/json

{
  "action": "revision",
  "note": "The total is cut off in the photo – please retake it."
}
```

```
HTTP/1.1 200 (10 ms)
Content-Type: application/json
```

```
{
  "_id": "6ac4dfd59a8d0ac9822daa78",
  "owner": "6ac4dfcfca6b2c5e871b50ed",
  "ownerName": "John Paul Dela Cruz",
  "entryId": "6ac4dfd59a8d0ac9822daa6f",
  "version": 1,
  "parentId": null,
  "latest": true,
  "kind": "file",
  "fileName": "sm-groceries.jpg",
  "mimeType": "image/png",
  "size": 1618,
  "url": "",
  "provider": "",
  "status": "Needs Revision",
}
```

```

    "reviewerName": "System Admin",
    "reviewedAt": "2026-10-06T11:47:34.076Z",
    "reviewNote": "The total is cut off in the photo – please retake it.",
    "submittedAt": "2026-10-06T11:47:33.644Z"
  }

```

5. Owner submits version 2 as a Google Drive link — request and response

```

POST /api/receipts HTTP/1.1
Authorization: Bearer <session token>
Content-Type: application/json

{
  "transactionId": "6ac4dfd59a8d0ac9822daa6f",
  "kind": "link",
  "url": "https://drive.google.com/file/d/sample-sm-groceries/view"
}

HTTP/1.1 201 (9 ms)
Content-Type: application/json

{
  "_id": "6ac4dfd69a8d0ac9822daa91",
  "owner": "6ac4dfcfca6b2c5e871b50ed",
  "ownerName": "John Paul Dela Cruz",
  "entryId": "6ac4dfd59a8d0ac9822daa6f",
  "version": 2,
  "parentId": "6ac4dfd59a8d0ac9822daa78",
  "latest": true,
  "kind": "link",
  "fileName": "",
  "mimeType": "",
  "size": 0,
  "url": "https://drive.google.com/file/d/sample-sm-groceries/view",
  "provider": "Google Drive",
  "status": "For Review",
  "reviewerName": "",
  "reviewedAt": null,
  "reviewNote": "",
  "submittedAt": "2026-10-06T11:47:34.088Z"
}

```

6. Admin verifies version 2 — request and response (excerpt)

```

POST /api/receipts/6ac4dfd69a8d0ac9822daa91/review HTTP/1.1
Authorization: Bearer <session token>
Content-Type: application/json

{
  "action": "verify",
  "note": ""
}

HTTP/1.1 200 (10 ms)
Content-Type: application/json

{
  "_id": "6ac4dfd69a8d0ac9822daa91",
  "version": 2,
  "status": "Verified",
  "reviewerName": "System Admin",
  "reviewedAt": "2026-10-06T11:47:34.094Z"
}

```

7. Pay a bill — the cascade's response (bill, generated expense entry, next monthly bill)

```

POST /api/bills/6ac4dfcfca6b2c5e871b50f8/pay HTTP/1.1

```

```
Authorization: Bearer <session token>
Content-Type: application/json
```

```
{
  "amount": 2450
}
```

```
HTTP/1.1 200 (10 ms)
Content-Type: application/json
```

```
{
  "bill": {
    "_id": "6ac4dfcfca6b2c5e871b50f8",
    "name": "Meralco Electricity",
    "category": "Utilities",
    "amount": 2450,
    "due": "2026-10-08",
    "paid": true,
    "paidOn": "2026-10-06",
    "paidAmount": 2450,
    "repeat": "monthly"
  },
  "transaction": {
    "_id": "6ac4dfd69a8d0ac9822daaa7",
    "type": "Expense",
    "category": "Utilities",
    "amount": 2450,
    "date": "2026-10-06",
    "note": "Bill payment: Meralco Electricity",
    "status": "Approved",
    "autoApproved": true
  },
  "nextBill": {
    "name": "Meralco Electricity",
    "due": "2026-11-08",
    "paid": false,
    "repeat": "monthly",
    "previousBill": "6ac4dfcfca6b2c5e871b50f8"
  }
}
```

8. Notifications created by the workflow (owner's inbox, newest first)

```
[payment] Payment recorded for "Meralco Electricity" – status auto-updated to Paid. Next bill scheduled for 2026-11-08.
[receipt-verified] Your receipt v2 for the expense of ₱1,845.50 (Food, 2026-10-06) was verified.
[receipt-revision] Your receipt v1 for the expense of ₱1,845.50 (Food, 2026-10-06) was sent back for revision: "The total is cut off in the photo – please retake it."
[reminder] "Meralco Electricity" is due in 2 days – ₱2,450.00.
[overdue] "Maynilad Water" is overdue by 1 day – ₱680.00 still unpaid.
[reminder] "Condo Rent" is due today – ₱14,000.00.
```

Admin's inbox:

```
[receipt] John Paul Dela Cruz submitted receipt v2 for an expense of ₱1,845.50 (Food, 2026-10-06).
[receipt] John Paul Dela Cruz submitted a receipt for an expense of ₱1,845.50 (Food, 2026-10-06).
```

7.5 Validation and Error Handling

Validation happens on the server for every write, whatever the client sends; the browser's own checks only make mistakes quicker to fix. Errors return the right HTTP status with a plain-language message, never a stack trace, and failed changes are logged. The table shows real responses captured from the system:

Table 34. Error handling — captured responses

Case	Request	Status	Message returned
Missing data	POST /api/bills	400	Bill name, category, due date, and a numeric amount are required.
Invalid data	POST /api/transactions	400	Date cannot be in the future.
Invalid file	POST /api/receipts	400	That file is not really a photo. Upload the receipt itself.
Duplicate / invalid transition	POST /api/bills/:id/pay	400	This bill is already marked as paid.
Duplicate data	POST /api/budgets	409	You already have a budget for "Food".
Failed approval (already decided)	POST /api/receipts/:id/review	409	This receipt was already decided, replaced or withdrawn.
Unauthorized action	POST /api/receipts/:id/review	403	You do not have permission to perform this action.
No session	GET /api/transactions	401	Not authenticated. Please log in.
Wrong password	POST /api/auth/login	401	Invalid email or password.

Table 35. How missing, invalid, duplicate and failed data are handled

Kind	Mechanism
Missing	Required fields checked in controllers and schemas → 400 with the name of what is missing. Forms keep what was typed.
Invalid	Types checked (text must be text, amounts must be numbers 0.01 – 1e12, dates must be real and not in the future, categories must match the type); uploads checked by type, size and first bytes; links must be https; malformed ids → 400; bodies over the limit → 413; malformed JSON → 400.
Duplicate	Unique indexes (email; budget per category) → 409. Atomic conditional updates stop double payments, double decisions and two simultaneous v2s (tested with 10 simultaneous requests: exactly one succeeds).
Failed	Multi-step workflows undo their first step if a later step fails (bill set back to unpaid; previous receipt version restored; previous entry version restored). Every refused or failed change is written to the audit log as Failed. The client gives up on a request after 60 s and says whether the change may still have been saved; a lost connection shows a message instead of signing the user out.

7.6 Integration Logs

Every action is recorded with a timestamp, the person or service, the action, a **status** (Success or Failed), a detail and the **reference id** of the record concerned, so the lines of one workflow run can be traced together. The log below was captured during the workflow in Section 7.4 (newest first):

Table 36. Captured integration / audit log — successful and failed entries

Time (PH)	User	Action	Status	Detail	Ref (last 8)
19:47:34	John Paul Dela Cruz	Failed: POST budgets	Failed	409 — You already have a budget for "Food".	—
19:47:34	John Paul Dela Cruz	Failed: POST transactions	Failed	400 — Date cannot be in the future.	—
19:47:34	John Paul Dela Cruz	Failed: POST bills	Failed	400 — Bill name, category, due date, and a numeric amount are required.	—
19:47:34	John Paul Dela Cruz	Failed: POST receipts/:id/review	Failed	403 — You do not have permission to perform this action.	822daa91
19:47:34	System Admin	Failed: POST receipts/:id/review	Failed	409 — This receipt was already decided, replaced or withdrawn.	822daa91
19:47:34	John Paul Dela Cruz	Failed: POST receipts	Failed	400 — That file is not really a photo. Upload the receipt itself.	—

Time (PH)	User	Action	Status	Detail	Ref (last 8)
19:47:34	John Paul Dela Cruz	Failed: POST bills/:id/pay	Failed	400 — This bill is already marked as paid.	871b50f8
19:47:34	John Paul Dela Cruz	Payment Recorded	Success	Meralco Electricity (6ac4dfcfa6b2c5e871b50f8) marked Paid, amount 2450. Triggered: expense log entry + notification + next monthly bill due 2026-11-08. Expense entry 6ac4dfd69a8d0ac9822daaa7.	871b50f8
19:47:34	System Admin	Receipt Verified	Success	v2 from John Paul Dela Cruz for the expense of ₱1,845.50 (Food, 2026-10-06).	822daa91
19:47:34	John Paul Dela Cruz	Receipt Resubmitted	Success	v2 for expense of ₱1,845.50 (Food, 2026-10-06): Google Drive link. Status set to For Review.	822daa91
19:47:34	System Admin	Receipt Revision Requested	Success	v1 from John Paul Dela Cruz for the expense of ₱1,845.50 (Food, 2026-10-06) — "The total is cut off in the photo — please retake it.".	822daa78
19:47:33	John Paul Dela Cruz	Receipt Submitted	Success	v1 for expense of ₱1,845.50 (Food, 2026-10-06): sm-groceries.jpg (2 KB). Status set to For Review.	822daa78
19:47:33	John Paul Dela Cruz	Expense Recorded	Success	Food: 1845.5 on 2026-10-06.	822daa6f
19:47:33	John Paul Dela Cruz	Failed Login	Failed	Incorrect password.	871b50ed
19:47:33	System Admin	Login	Success	Successful authentication (Admin)	—
19:47:33	John Paul Dela Cruz	Login	Success	Successful authentication (User)	—
19:47:28	Reminder Service	Reminder Sweep	Success	2026-10-06: scanned 3 unpaid bill(s) within 3 day(s); raised 3 alert(s) (1 overdue).	—
19:47:27	System	Seed	Success	Sample data initialized for demo.	—

FS

FinTrack Stark

Integration System

OVERVIEW

Dashboard

REVIEW

Receipt Review 2

SYSTEM

Notification Log 2

Audit Log

User & Role Mgmt

System Settings

Settings

Dark mode

SA System Admin Admin

Log out

Audit Log

fintrackstark/audit

Tuesday, October 6, 2026

Audit Log

Record of key actions — sign-ins, bills, payments, entries, receipts and reviews — with failed and refused actions marked, and the id of the record each one concerned.

All 20

Success 12

Failed 8

Timestamp	User	Action	Status	Detail	Reference
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST budgets	Failed	409 — You already have a budget for "Food".	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST transactions	Failed	400 — Date cannot be in the future.	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST bills	Failed	400 — Bill name, category, due date, and a numeric amount are required.	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST receipts/:id/review	Failed	403 — You do not have permission to perform this action.	822daa91
10/6/2026, 7:47:34 PM	System Admin	Failed: POST receipts/:id/review	Failed	409 — This receipt was already decided, replaced or withdrawn.	822daa91
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST receipts	Failed	400 — That file is not really a photo. Upload the receipt itself.	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST bills/:id/pay	Failed	400 — This bill is already marked as paid.	871b50f8
10/6/2026, 7:47:33 PM	John Paul Dela Cruz	Failed Login	Failed	Incorrect password.	871b50ed

Figure 27. Audit Log screen filtered to Failed: refused and failed integration actions with their error messages

8. Security, Governance, and Risk Management

8.1 Role-Based Access Control Matrix

Access is enforced on the server for every request (`requireAuth`, `requireRole`, and ownership checks in each controller). The interface also hides what a role cannot use, but it is not the control.

Table 37. RBAC matrix (✓ allowed, ✗ refused, Own = only their own records)

Function	Visitor	User	Admin	Reminder service
Register / log in	✓	—	—	—
Bills, entries, budgets, notes	✗	Own	✗ (no wallet)	—
Notifications	✗	Own	Own	writes
Pay a bill	✗	Own	✗	✗
Attach a receipt	✗	Own entries	✗	✗
Open an uploaded receipt	✗	Own	Submitted receipts	✗
View review queue; verify / reject / request revision	✗	✗	✓	✗
View entry version history	✗	Own	✗	✗
View audit / integration log	✗	✗	✓	writes only
List users; change roles; delete accounts; reset passwords	✗	✗	✓ (not self)	✗
System settings (reminders, budget warning, sessions)	✗	✗	✓	reads lead time
Raise bill reminders	✗	✗	✗	✓
Health check (/api/health)	✓	✓	✓	—

8.2 Least Privilege and Separation of Duties

- **Registration always creates a User.** A role is never chosen at sign-up; only an Admin can make someone an Admin, and the retired Reviewer role cannot be assigned (ST-03).
- **The Admin keeps no wallet.** Every personal-finance endpoint refuses an Admin, so the person who judges receipts never submits any — the strongest form of "nobody reviews their own evidence" (RT-05, ST-04).
- **The Admin sees only what a review needs** — a submitted receipt and the one entry it proves — never a person's wallet, bills, budgets or notes (RT-31, ST-04). The server also refuses a decision on one's own receipt, in case a User is later promoted.
- **No self-escalation:** an Admin cannot change their own role or delete their own account, and the last Admin cannot be demoted or deleted.
- **Bills, payments and entries are owner-only** — nobody, Admin included, can pay or change another person's bill or entry.
- **Temporary passwords issued by an Admin** open a session that can do nothing but set a new password, so the Admin never keeps working access to the account (RT-06).
- **The database user** used by the deployed API has readWrite on its own database only — no admin rights on the cluster.
- **The reminder service** may write only `lastRemindedOn`, notifications and its own audit entries.

8.3 Data Privacy and Protection

Table 38. Data protection controls

Data	Protection
Passwords	Never stored; script hash with a random salt per password (old bcrypt hashes are upgraded on next sign-in). Login is rate-limited per address and per account, and an unknown email takes as long to refuse as a wrong password, so attackers cannot tell which emails exist.
Sessions	32 random bytes; only a SHA-256 hash is stored, so a leaked database gives no working session. Expire after the session length set by the Admin (8 hours by default); ended by logout, password change (all devices) or account deletion. Kept in sessionStorage, which is cleared when the tab closes.
Financial records and notes	Readable only by their owner (NFR-002), checked on the server for every request.
Receipts	Owner and the reviewing Admin only; files are served with the session token (never a public URL), never cached by the browser, and type-checked on upload. Links are stored as links — the file stays in the owner's Drive.
In transit	HTTPS between browser and server; TLS between server and MongoDB Atlas; HSTS tells browsers to refuse plain HTTP.
In the browser	Content-Security-Policy (scripts only from the site), X-Frame-Options DENY, nosniff, no-referrer, restrictive Permissions-Policy. React escapes all text, so names such as are shown, not run.
Logs	No passwords or tokens are logged; database connection strings are printed with the password masked.
Personal data policy	A Privacy Policy and Terms page explain what is collected, why, and who can see it, in line with the Data Privacy Act of 2012 (RA 10173). Only sample data is used in the project (spec §21.7–8).

8.4 Audit Trail

The audit log records sign-ins and failed sign-ins; account creation, profile and password changes; role changes, password resets and account deletions; bills created, edited, deleted and paid; entries recorded, edited and deleted; receipts submitted, resubmitted, verified, rejected and sent back; reminder sweeps; and every refused or failed change, with its error. Each line has a timestamp, the person or service, the action, the status, the detail and the record reference (see Section 7.6). The Admin Console summarises the week's activity, including failed actions.

FS

FinTrack Stark

Integration System

OVERVIEW

Dashboard

REVIEW

Receipt Review

2

SYSTEM

Notification Log

2

Audit Log

2

User & Role Mgmt

System Settings

Settings

Audit Log

fintrackstark/audit

Tuesday, October 6, 2026

Audit Log

Record of key actions — sign-ins, bills, payments, entries, receipts and reviews — with failed and refused actions marked, and the id of the record each one concerned.

All

20

Success

12

Failed

8

TIMESTAMP	USER	ACTION	STATUS	DETAIL	REFERENCE
10/6/2026, 7:47:35 PM	System Admin	Login	Success	Successful authentication (Admin)	—
10/6/2026, 7:47:35 PM	John Paul Dela Cruz	Login	Success	Successful authentication (User)	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST budgets	Failed	409 — You already have a budget for "Food".	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST transactions	Failed	400 — Date cannot be in the future.	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST bills	Failed	400 — Bill name, category, due date, and a numeric amount are required.	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST receipts/id/review	Failed	403 — You do not have permission to perform this action.	822daa91
10/6/2026, 7:47:34 PM	System Admin	Failed: POST receipts/id/review	Failed	409 — This receipt was already decided, replaced or withdrawn.	822daa91
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST receipts	Failed	400 — That file is not really a photo. Upload the receipt itself.	—
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Failed: POST bills/id/pay	Failed	400 — This bill is already marked as paid.	871b50f8
10/6/2026, 7:47:34 PM	John Paul Dela Cruz	Payment Recorded	Success	Meralco Electricity (6ac4dfcfa6b2c5e871b50f8) marked Paid, amount 2450. Triggered: expense log entry + notification + next monthly bill due 2026-11-08. Expense entry 6ac4dfd69a8d0ac9822daa7.	871b50f8
10/6/2026, 7:47:34 PM	System Admin	Receipt Verified	Success	v2 from John Paul Dela Cruz for the expense of P1,845.50 (Food, 2026-10-06).	822daa91
10/6/2026, 7:47:34 PM	John Paul	Receipt Verified	Success	v2 for expense of P1,845.50 (Food, 2026-10-06): Google Drive link. Status set to For	871b50f8

Figure 28. Audit Log: every action with status and record reference

8.5 Integration Ownership and Governance

Table 39. Data and module ownership

Data / module	Owning module (writer)	Others may	Accountable team role
Users, roles, sessions	Auth API, Users API	Read own profile	Backend / Integration Developer; QA / Security Lead
Bills	Bills API	Reminder service: set lastRemindedOn only	Backend / Integration Developer
Transactions	Transactions API, payment workflow	Receipts: read the entry	Backend / Integration Developer
Receipts	Receipts API (owner: submit; Admin: decide)	—	Backend / Integration Developer
Notifications	Notification service, reminder service	Addressee: mark read	Backend / Integration Developer
Audit log	Audit service, failure-log middleware, reminder service	Admin: read	QA / Security Lead
Event bus (SSE)	events service	Any module: publish a topic	Backend / Integration Developer
System settings	Settings API (Admin)	API services and reminder service: read	Project Manager / Systems Analyst
API contract (routes)	Routes layer	Client: call	Project Manager / Systems Analyst
Deployment, backups	Render service, backup/restore scripts	—	Project Manager; QA / Security Lead
User interface	React client	—	UI/UX and Multimedia Lead

Governance rules: (1) a collection is changed only by its owning module; (2) cross-module effects go through services (audit, notifications, events), never by one controller writing another module's fields; (3) every change to a route or schema is accompanied by an automated test; (4) changes reach production only through GitHub, which triggers the Render build; (5) a backup is taken before any deployment that changes a schema.

8.6 Risk Register

Table 40. Risk register (likelihood and impact: Low / Medium / High)

#	Risk	Likelihood	Impact	Level	Control	Monitoring
R1	Unauthorized access to another user's records	Low	High	Medium	Server-side owner and role checks on every route; ST-01–ST-04	Failed (403) lines in the audit log; security tests on every change
R2	Password guessing / credential stuffing	Medium	High	High	Rate limits per IP and per account; scrypt; same timing for unknown emails	Failed Login lines; Admin Console failed-actions bar
R3	Malicious or oversized upload	Medium	Medium	Medium	Type allow-list, first-byte check, 2 MB limit, files never executed, CSP	Failed: POST receipts lines (ET-06)
R4	Double payment or double decision from simultaneous requests	Medium	Medium	Medium	Atomic conditional updates; rollback on partial failure	409 lines; ET-05, ET-07, RT-02
R5	Data loss (database failure, mistake)	Low	High	Medium	npm run backup (incl. files) before deploys and weekly; tested restore	Backup files dated; restore drill each release (RT-34)
R6	Restore run against the live database by mistake	Low	High	Medium	Restore refuses to run without --yes and prints the target first	Operator checklist; RT-34
R7	Hosting sleeps or is down (free tier)	High	Medium	High	Client shows clear messages and retries; reminders catch up on wake; data in Atlas	/api/health; Render dashboard
R8	Leaked database credentials	Low	High	Medium	Least-privilege DB user; secrets only in Render env vars; masked in logs	Atlas access logs; rotate password on suspicion
R9	Shared-database conflict between API and reminder service	Low	Medium	Low	Field ownership; atomic per-bill claim	Reminder Sweep audit lines; IT-06, RT-23
R10	Sensitive data in receipts seen by the wrong person	Low	High	Medium	Owner + reviewing Admin only; Admin sees the receipt and one entry, never a wallet; no public URLs	ST-04; Failed lines for 403 on files
R11	Regression after a change	Medium	Medium	Medium	68 automated tests run before every push	Test report (Section 9.6)
R13	A setting is changed to an unsafe value (e.g. very long sessions)	Low	Medium	Low	Limits enforced on the server (sessions ≤ 24 h, lead time ≤ 14 days); Admin only	"Settings Changed" lines in the audit log (RT-36)
R12	Team members cannot explain the AI-assisted code at the defense	Medium	High	High	Code walkthroughs; this document; comments in the code explain each decision	Practice defense with the demo script (Section 11)

9. Testing and Evaluation

9.1 Test Plan

Table 41. Test plan

Item	Approach
Objective	Show that every functional requirement works, that modules exchange data correctly, that bad input and unauthorized access are refused cleanly, and that one complete workflow runs end to end.
Method	Automated black-box tests with Playwright 1.63 driving Google Chrome through the real user interface, plus direct HTTP calls to the API for access-control and error cases. Each case is named by category and requirement (e.g. FT-03 (FR-003)).
Environment	The real API server started by the test runner, a dedicated database <code>fintrack_stark_test</code> wiped and re-seeded before every run (never the live data), Chrome at 1440 × 900 and phone sizes 320–390 px.
Categories	Functional (FT), Integration (IT), Error-handling (ET), Security / access control (ST), End-to-end (E2E), and Regression (RT) cases kept from three pre-launch audits.
Entry criteria	Code builds; the seed data loads; the API health check answers.
Exit criteria	All cases pass on two consecutive runs; no case is skipped.
Defect handling	Every defect found in audits gets a regression case (RT-xx) that failed before the fix and passes after it.

Table 42. Test coverage against the specification's minimum (§16)

Test type	Required minimum	Implemented	Passed
Functional test cases	8	12	12
Integration test cases	5	8	8
Error-handling test cases	5	7	7
Security / access-control test cases	3	4	4
End-to-end test scenario	1	2	2
Regression cases (additional)	—	37	37

9.2 Functional Test Cases

Table 43. Functional test cases

ID	Test case	Req.	Expected result	Result
FT-01	A registered user can sign in through the form	FR-001	Valid credentials open the dashboard; the session is created	Passed
FT-02	A user can create a bill record	FR-002	Bill saved and shown in its category and list	Passed
FT-03	A user can record an income/expense entry	FR-003	Entry recorded and counted in the balance at once	Passed
FT-04	The system tracks entry versions	FR-004	Editing creates v2 and keeps v1 as an earlier version	Passed
FT-05	A user can correct and delete their own entries	FR-015	Owner can edit and delete; deleted entry stops counting and stays in history	Passed
FT-06	Notes are stored per entry	FR-006	Note saved against the entry and shown with it	Passed

ID	Test case	Req.	Expected result	Result
FT-07	The dashboard summarises balance, bills and budgets	FR-009	Dashboard shows balance, bills and budget figures that match the data	Passed
FT-08	Important actions are written to the audit log	FR-010	Login and other actions appear with timestamp, user, action, status, detail, reference; Failed filter works	Passed
FT-09	Every screen has its own address		Every screen opens from its own address	Passed
FT-10	Addresses survive a reload, and the browser's own buttons work		Reload keeps the screen; Back/Forward work	Passed
FT-11	A receipt photo can be attached while recording an entry	FR-003	Photo attached on the form; entry saved; receipt shows "For review" and opens	Passed
FT-12	The Admin can verify, reject or send back a receipt from the review page	FR-005	Admin opens the receipt with its entry and verifies it; it moves to "Decided by you"	Passed

9.3 Integration Test Cases

Table 44. Integration test cases

ID	Test case (data exchange tested)	Expected result	Result
IT-01	Paying a bill cascades into an expense, a notification, and an audit entry	One payment creates the Paid bill, an auto expense entry, a notification and an audit line	Passed
IT-02	A notification reaches an open screen without a refresh	A notification reaches an open screen (toast + badge) without reloading	Passed
IT-03	Notifications reach only the person they are about	Notifications reach only the bill's owner	Passed
IT-04	A recorded entry reaches the dashboard at once, without a refresh	A new entry updates an open dashboard without reloading	Passed
IT-05	Recorded expenses are counted against the matching budget	Budget and Reports screens show the same spent amount as the data	Passed
IT-06	The reminder service raises overdue alerts on its own schedule	Separate reminder process raises one overdue alert for the owner only; second run adds none	Passed
IT-07	A receipt moves through review: submitted, sent back, resubmitted as v2, verified	Submit → Admins alerted → revision (note on entry) → v2 → verified; all steps audited	Passed
IT-08	Changing the figures of an entry sends its verified receipt back for review	Note-only edit keeps Verified; amount change sends the receipt back for review	Passed

9.4 Error-Handling Test Cases

Table 45. Error-handling test cases

ID	Test case	Expected result	Result
ET-01	Missing required fields are rejected with a usable message	400 with a usable message for each missing field	Passed
ET-02	Invalid amounts are rejected at the boundary	Zero, negative, non-numeric and huge amounts refused (400)	Passed
ET-03	Duplicate data is refused and the reason is explained	Duplicate email and duplicate budget refused (409) with the reason	Passed
ET-04	A malformed record id returns 400, not a crash or a hang	Malformed ids return 400 — no crash, no hang	Passed
ET-05	An invalid workflow transition is refused	Paying twice, editing an old version, deleting twice refused	Passed
ET-06	Invalid uploads are refused with a reason, and nothing is saved	Wrong type, fake file, > 2 MB, http/javascript links, deleted entry refused; nothing saved	Passed
ET-07	A failed review is refused cleanly and written to the log as Failed	Missing note → 400; second decision → 409; verified receipt cannot be replaced; refusal logged as Failed with ref	Passed

Security / access-control test cases

Table 46. Security and access-control test cases

ID	Test case	Expected result	Result
ST-01	Protected endpoints refuse unauthenticated callers	Every protected endpoint returns 401 without a token	Passed
ST-02	A user cannot reach another user's records	A user gets 403/404 for another user's records	Passed
ST-03	Privileges cannot be escalated from a lower role	A User cannot use Admin functions; sign-up cannot request a role; invalid and retired roles refused	Passed
ST-04	Receipts are seen only by their owner and the Admin, and only the Admin decides	Receipt files: owner and Admin only (never another User); only the Admin decides; an Admin cannot record entries	Passed

9.5 End-to-End Test Scenario

E2E-02 — a receipt goes from submission to final approval. Two browsers act as two people at the same time; no step uses an API shortcut, and each person sees the other's actions without reloading.

Table 47. E2E-02 steps

#	Who	Action	Expected and observed
1	User	Log Income/Expense: Expense ₱2,345, tick "Attach a receipt", choose a photo, Save Entry	Entry recorded; toast "receipt sent for review"
2	Admin	Receipt Review page (already open)	The ₱2,345.00 receipt appears live, without reloading
3	Admin	Review receipt → note "Photo is blurry — please retake it" → Request revision	Toast "Sent back for revision"
4	User	My Entries	Chip shows "Receipt v1 · Needs revision"; dialog shows the note
5	User	Choose a new photo → Submit for Review	Chip changes to "Receipt v2 · For review"
6	Admin	Card shows "version 2" → Review receipt → Verify	Toast "Receipt verified"
7	User	(no action)	Chip changes to "Receipt v2 · Verified" live; inbox has a receipt-verified notification

Result: Passed (3.5 s). A second scenario, **E2E-01**, covers the money side: a new account registers from the landing page, sets a budget, records an expense, corrects it to v2, and the corrected figure reaches the dashboard, the budget and the reports, with every step in the audit log (Passed).

9.6 Test Results Summary

The full suite was run on 10/6/2026, 20:30:32 (Philippine time): **70 test cases, 70 passed, 0 failed, 0 skipped**, in 102 seconds. The suite had also passed twice in a row before this run, the exit criterion. Every case is listed with its result in Appendix C.

Table 48. Results summary

Category	Cases	Passed	Failed
FT	12	12	0
IT	8	8	0
ET	7	7	0

Category	Cases	Passed	Failed
ST	4	4	0
E2E	2	2	0
RT (regression)	37	37	0
Total	70	70	0

Defects found and fixed during testing (selected). Three pre-launch audits found, among others: double payment when a button was clicked twice; amounts such as true accepted as ₱1; a restore script that could wipe a database without confirmation; the landing page scrolling sideways on 320–360 px phones; an empty ₱0.00 dashboard shown on slow connections before data arrived; requests that could wait forever; and an Admin able to read another person's entry history. Each now has a regression case (RT-01 to RT-35) that failed before its fix.

10. Deployment, Backup, and Recovery

10.1 Deployment Environment

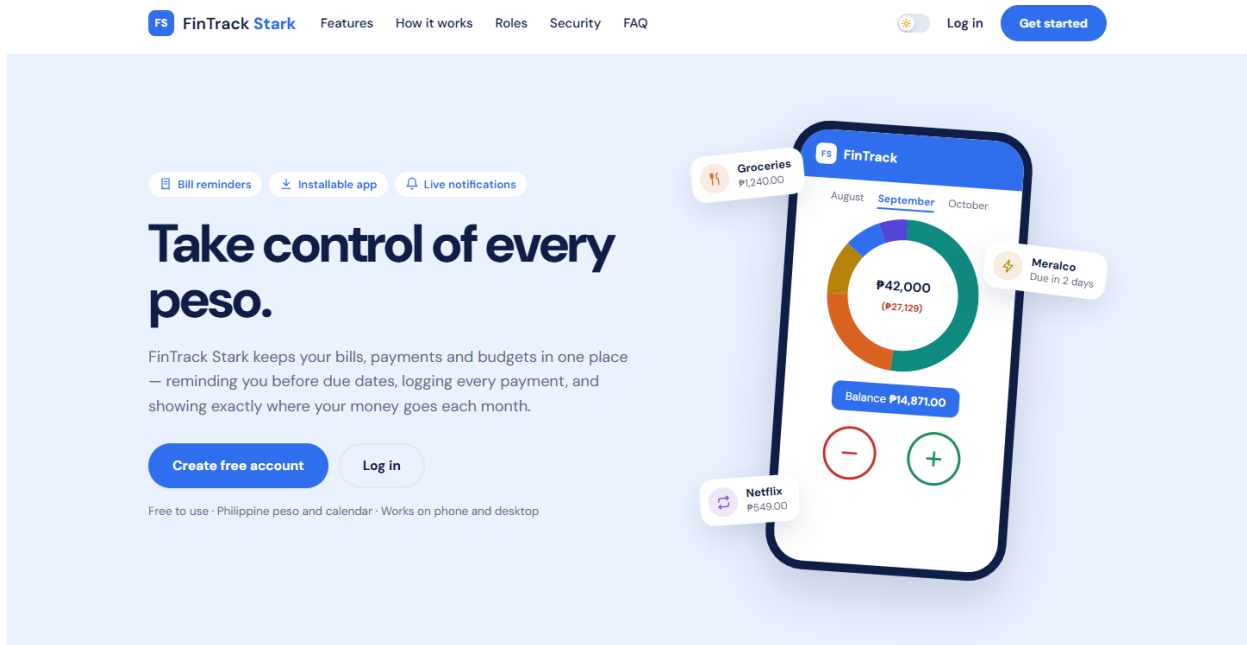
Table 49. Environments

Environment	Where	Details
Production	Render Web Service (Singapore) + MongoDB Atlas M0 (AWS Hong Kong)	https://fintrack-stark.onrender.com — one Node.js service serves the API and the built React app; env vars <code>NODE_ENV=production</code> , <code>TRUST_PROXY=1</code> , <code>MONGO_URI</code> ; database user <code>fintrack_app</code> with <code>readWrite</code> on <code>fintrack_stark</code> only; the first Admin was created with <code>npm run create-admin</code> (the demo seed refuses to run in production).
Local / demo	Developer laptop: Node.js 24 + MongoDB 8.3 (local service)	<code>npm run seed</code> loads sample data; <code>npm run dev</code> starts the API (port 4000), the client (port 5173) and the separate reminder worker together.
Test	Same laptop, database <code>fintrack_stark_test</code>	Created, seeded and used only by the automated tests.

10.2 Deployment Evidence

The deployed prototype is available at <https://fintrack-stark.onrender.com>. On the free tier the service sleeps after a period without visitors; the first visit afterwards takes up to about a minute while it starts. The screenshots below were taken from the live site and from the API's health endpoint.

Before submission: deploy the latest commit (git push origin main), open the live site, and retake these two screenshots so they show the current version with receipt review. [ILAGAY DITO kung may bagong screenshot]



FEATURES

Figure 29. Deployed system — landing page at <https://fintrack-stark.onrender.com>

🔒 https://fintrack-stark.onrender.com/api/health

```
{
  "status": "ok",
  "service": "fintrack-stark-api",
  "db": "connected",
  "reminders": {
    "lastSweep": "2026-10-06"
  },
  "time": "2026-10-06T11:48:09.551Z"
}
```

Figure 30. Deployed system — /api/health answering from Render with the database connected

🔒 http://localhost:4100/api/health

```
{
  "status": "ok",
  "service": "fintrack-stark-api",
  "db": "connected",
  "reminders": {
    "lastSweep": "2026-10-06"
  },
  "time": "2026-10-06T11:48:08.444Z"
}
```

Figure 31. Browser evidence of the API (local server): health endpoint JSON

10.3 Backup Plan

Table 50. What is backed up, how and when

Asset	How	When / where
Source code	Git; every change is pushed to GitHub (jeypydc02-ui/SIA-Final-Project); every member can clone a full copy	On every commit; GitHub + members' laptops
Database	npm run backup writes every collection to one timestamped JSON file, including uploaded receipt files (as base64). Built on the app's own models, so it runs anywhere Node runs	Before every deployment that changes data, and weekly; kept in backups/ and copied to the group's shared drive
Receipt files	Uploaded files are inside the database backup; linked files stay in the owner's cloud storage (Google Drive / OneDrive / Dropbox), which keeps its own copies	With the database backup
Configuration / secrets	Environment variables recorded in Render; the Atlas password kept by the project manager, never in the repository	When changed
Documentation	This document, diagrams and screenshots in the repository's docs/ folder and the group's shared drive	Each version

Backup run, captured from the terminal

```
> npm run backup -- --out backups/fintrack-stark-2026-10-06.json
[db] connected to mongodb://127.0.0.1:27017/fintrack_stark_docs
[backup] wrote backups/fintrack-stark-2026-10-06.json (32.2 KB)
  3  users
  8  bills
 11  transactions
  4  budgets
  9  notifications
  5  comments
 20  auditLogs
  7  receipts
  0  settings
```

10.4 Recovery Plan

Table 51. Failure scenarios and recovery steps

Failure	Recovery steps	Target time
Database lost or corrupted	1. Point MONGO_URI at the (new) database. 2. <code>npm run restore -- <file></code> — it shows the target and changes nothing. 3. Check the target, run again with <code>--yes</code> . 4. Everyone signs in again (sessions are not restored).	≤ 30 minutes
Bad deployment	Roll back to the previous deploy in the Render dashboard (or revert the commit and push); restore a backup if data changed.	≤ 15 minutes
Render outage	Run locally (<code>npm run build && npm start</code>) against Atlas for the demo; or create a new Render service from the same GitHub repository.	≤ 30 minutes
Atlas credentials leaked	Change the <code>fintrack_app</code> password in Atlas, update MONGO_URI in Render, redeploy; review the audit log.	≤ 15 minutes
Admin locked out	<code>npm run create-admin</code> against the database creates a new Admin; or another Admin issues a temporary password.	≤ 10 minutes
Reminder sweep failed	Logged as Failed in the audit log; the API retries after 5 minutes; <code>npm run reminder</code> : once runs it by hand. Bills already reminded that day are never alerted twice.	Automatic

Recovery drill, captured from the terminal (restore into a new database)

```
> npm run restore -- backups/fintrack-stark-2026-10-06.json
[restore] source: backups/fintrack-stark-2026-10-06.json
[restore] taken: 2026-10-06T11:48:10.730Z
[restore] target: mongodb://127.0.0.1:27017/fintrack_stark_recovery
[restore] Nothing was changed. A restore DELETES everything in the target
database and replaces it with the backup. Check the target above,
then run the same command again with --yes to go ahead.

> npm run restore -- backups/fintrack-stark-2026-10-06.json --yes
[restore] source: backups/fintrack-stark-2026-10-06.json
[restore] taken: 2026-10-06T11:48:10.730Z
[restore] target: mongodb://127.0.0.1:27017/fintrack_stark_recovery
[db] connected to mongodb://127.0.0.1:27017/fintrack_stark_recovery
   3 users
   8 bills
  11 transactions
   4 budgets
   9 notifications
   5 comments
  20 auditLogs
   7 receipts
   0 settings
[restore] complete — indexes rebuilt.
```

11. System Demonstration Guide

11.1 Demo Script

Use two browser windows (one normal, one incognito) so that two people can be signed in at once. Run `npm run seed` beforehand on the local database for a clean start (about 15 minutes in total).

Table 52. Demo script

#	Account	Do	Point out (spec item)
1	—	Open the landing page; show it on a phone-sized window; install as an app	Mobile/web; PWA
2	User (jp@)	Log in; walk through the Home dashboard	Login page; Dashboard (§14.1–2)
3	User	Bill Categories → Utilities	Project List / Details (§14.3–4)
4	User	Bill Reminders → Mark Paid on Meralco	Workflow automation / webhook: Paid + expense + notification + audit + next bill (I2)
5	User	Log Income/Expense: expense with a receipt photo	Asset submission form (§14.5); status For Review automatically
6	Admin (window 2)	Admin Console shows the new receipt without refreshing; open Receipt Review	Messaging / live events (I4); Review page (§14.7)
7	Admin	Request revision with a note	Review and Approval workflow
8	User	Inbox notification; My Entries → Receipt v1 · Needs revision → attach v2 (Drive link)	Notification log (§14.9); version tracking; external storage link (I6)
9	Admin	Verify v2	Final approval
10	User	Notes / Feedback → On Entries shows the Admin's review note; Revision History shows v1 → v2 of an entry	Comment page (§14.8); revision history (§14.6)
11	User	Try to pay the same bill again	Error handling: refused
12	Admin	Admin Console; Audit Log → Failed filter	Audit log (§14.10); integration logs (§7.6)
13	Admin	User & Role Mgmt; then open /bills as Admin — refused: an Admin keeps no wallet	User and role management (§14.12); least privilege, separation of duties
14	Admin	System Settings: change the reminder lead time; show the "Settings Changed" audit line	Admin manages settings (§12); audit trail
15	User	Reports	Reports page (§14.11)
16	Terminal	<code>npm run reminder:once</code> , <code>npm run backup</code> , <code>npm run restore</code> (without --yes)	Reminder service (I5); backup and recovery

11.2 Demo Accounts

Table 53. Demo accounts (local sample data only — the seed refuses to run in production)

Role	Email	Password	Name
User	jp@fintrackstark.app	demo123	John Paul Dela Cruz
User	arvy@fintrackstark.app	demo456	Arvy Karlson Dabasol
Admin	admin@fintrackstark.app	admin123	System Admin

On the deployed system, accounts are created through sign-up; the Admin account was created with `npm run create-admin` and can make another account an Admin from User & Role Management. An Admin keeps no wallet, so team members use a separate User account for their own records.

11.3 Sample Data

- **Bills (7):** Meralco Electricity (monthly), Maynilad Water (overdue), PLDT Home Fibr (monthly), Condo Rent (due today, monthly), Netflix Subscription (paid), BPI Credit Card, and Globe Postpaid for the second user.
- **Entries (9):** salary, groceries, transport, a freelance payment, the Netflix bill payment, a weekly market run, an allowance for the second user, and an airport-transfer expense corrected from ₱2,400 (v1) to ₱1,650 (v2).
- **Budgets (4):** Food ₱6,000, Transport ₱3,500, Utilities ₱5,000, Subscription ₱1,000.
- **Receipts (5):** two Verified (Google Drive and OneDrive links), one uploaded photo For Review, and one chain v1 Needs Revision → v2 For Review (Dropbox).
- **Notes:** a correction note, the Admin's revision note, and a personal savings plan.
- **Logs and notifications:** a welcome notification, and the reminder service's first sweep, which raises real reminders when the server starts.

12. Limitations and Future Enhancements

12.1 System Limitations

- **No real payment integration.** Payments are recorded, not made; the system does not connect to banks or e-wallets.
- **Notifications are in-app only.** There is no e-mail or SMS; a person who does not open the app does not see a reminder. Password recovery needs an Admin.
- **No native mobile app.** The PWA installs and opens offline, but cannot show push notifications while closed.
- **Unverified entries still count.** Receipt review checks evidence after the fact; it does not block an entry from the balance.
- **Receipts are compared by a person.** There is no automatic reading of totals or dates (OCR); uploads are limited to 2 MB.
- **Single API instance.** Live updates use an in-process event bus; running several API processes would need a shared channel.
- **Free hosting.** The service sleeps when idle (about a minute to wake), and the free database tier has no automatic backups, so backups are manual.
- **Audit log is not tamper-proof.** It cannot be changed through the application, but a database administrator could alter it.
- **Tested in Chrome on desktop and emulated phones;** not yet on Safari/iPhone hardware or under real multi-user load in production.

12.2 Recommended Enhancements

- E-mail/SMS reminders and web push notifications (e.g. via a messaging provider) so reminders reach people who are not in the app.
- OCR-assisted receipt review: read the total and date from the photo and flag mismatches for the Admin.
- Direct cloud storage integration (Google Drive / OneDrive APIs) instead of pasted links, and larger files in object storage.
- Import of bank or e-wallet statements (CSV) as an ETL integration, with matching against recorded entries.
- Shared household wallets, where several members record into one budget; and an optional separate Reviewer role for larger organisations.
- Native Android/iOS client sharing the same API.
- Paid hosting tier with automated database backups, monitoring and alerting; a message broker (or MongoDB change streams) for multi-instance live updates.
- Analytics: month-on-month trends, forecasts of upcoming bills, and savings goals.
- Tamper-evident audit log (hash-chained entries).

13. Individual Contribution

Fill in each member's actual role and specific contribution. Suggested roles from the specification (§17): Project Manager / Systems Analyst; UI/UX and Multimedia Lead; Backend / Integration Developer; QA / Security Lead; Documentation and Presenter.

Table 54. Individual contribution

Member name	Role	Specific contribution
Cain, Akhemee Yujinn	[ILAGAY DITO]	[ILAGAY DITO]
Dabasol, Arvy Karlson	[ILAGAY DITO]	[ILAGAY DITO]
Dela Cruz, John Paul	[ILAGAY DITO]	[ILAGAY DITO]
Heray, Neezar Ivan	[ILAGAY DITO]	[ILAGAY DITO]
Libaton, Jerico	[ILAGAY DITO]	[ILAGAY DITO]

14. References

- Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture*. O'Reilly Media.
- Nygard, M. (2011). *Documenting Architecture Decisions*.
<https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- ISO/IEC 25010:2011. *Systems and software engineering — System and software quality models*.
- The Open Group. *The TOGAF Standard* (business, data, application and technology architecture domains). <https://www.opengroup.org/togaf>
- OWASP Foundation. *OWASP Top 10 and Cheat Sheet Series* (Authentication, Session Management, File Upload, Password Storage). <https://cheatsheetseries.owasp.org>
- Republic Act No. 10173 — *Data Privacy Act of 2012* (Philippines). National Privacy Commission. <https://privacy.gov.ph>
- MDN Web Docs. *Using server-sent events; Content-Security-Policy; Progressive web apps*. <https://developer.mozilla.org>
- WHATWG. *HTML Living Standard — Server-sent events*.
<https://html.spec.whatwg.org/multipage/server-sent-events.html>
- React documentation. <https://react.dev>
- React Router documentation. <https://reactrouter.com>
- Express documentation. <https://expressjs.com>
- Node.js documentation — crypto (scrypt, randomBytes, timingSafeEqual).
<https://nodejs.org/api/crypto.html>
- MongoDB documentation and MongoDB Atlas documentation.
<https://www.mongodb.com/docs>
- Mongoose documentation. <https://mongoosejs.com/docs>
- Render documentation — Web Services. <https://render.com/docs>
- Playwright documentation. <https://playwright.dev>
- Mermaid documentation (diagrams in this document). <https://mermaid.js.org>

15. Appendices

Appendix A — Complete API Endpoint List

Table 55. All 42 API endpoints

Method	Path	Access
GET	/api/audit-log	Admin
POST	/api/auth/login	Anyone (rate-limited)
POST	/api/auth/register	Anyone (rate-limited)
GET	/api/auth/me	Signed in
PUT	/api/auth/me	Signed in
PUT	/api/auth/me/password	Signed in
POST	/api/auth/logout	Signed in
GET	/api/bills	User
POST	/api/bills	User
PUT	/api/bills/:id	User
DELETE	/api/bills/:id	User
POST	/api/bills/:id/pay	User
GET	/api/budgets	User
POST	/api/budgets	User
PUT	/api/budgets/:id	User
DELETE	/api/budgets/:id	User
GET	/api/comments	User
POST	/api/comments	User
PUT	/api/comments/:id	User
DELETE	/api/comments/:id	User
GET	/api/events	Signed in
GET	/api/notifications	Signed in
PUT	/api/notifications/read-all	Signed in
PUT	/api/notifications/:id/read	Signed in
GET	/api/receipts	User
POST	/api/receipts	User
GET	/api/receipts/review	Admin
GET	/api/receipts/:id/file	Signed in
POST	/api/receipts/:id/review	Admin
GET	/api/settings	Admin
PUT	/api/settings	Admin
GET	/api/transactions	User
POST	/api/transactions	User
GET	/api/transactions/:id/versions	User
PUT	/api/transactions/:id	User
DELETE	/api/transactions/:id	User
GET	/api/users	Admin

Method	Path	Access
PUT	/api/users/:id/role	Admin
POST	/api/users/:id/reset-password	Admin
DELETE	/api/users/:id	Admin
GET	/api/sync	Signed in
GET	/api/health	Anyone

Appendix B — Additional Screenshots

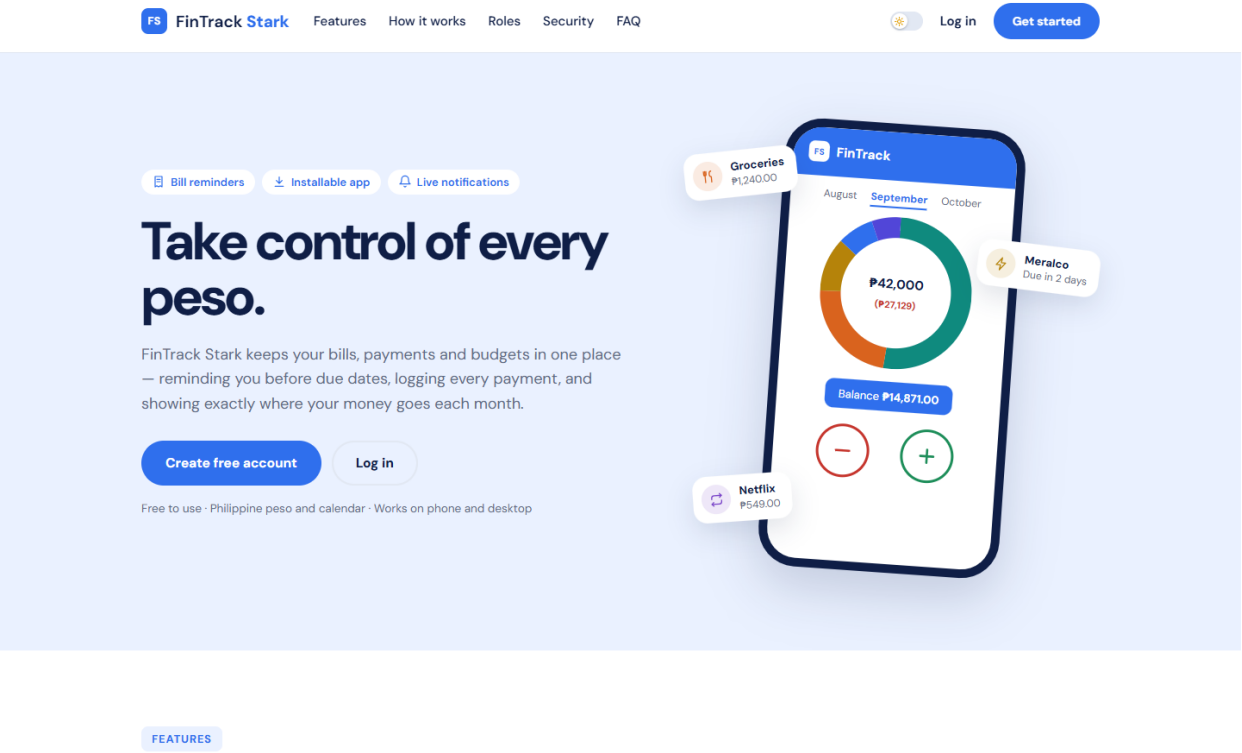


Figure 32. Landing page

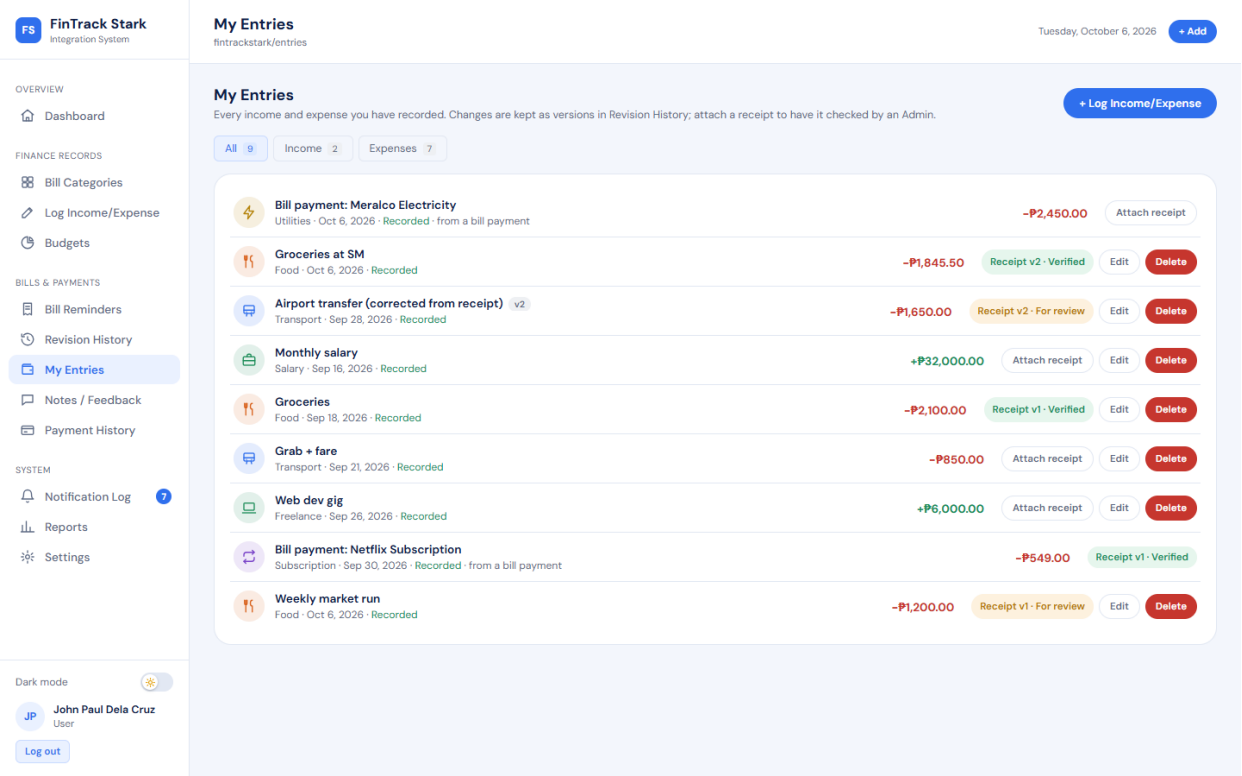


Figure 33. My Entries with receipt statuses

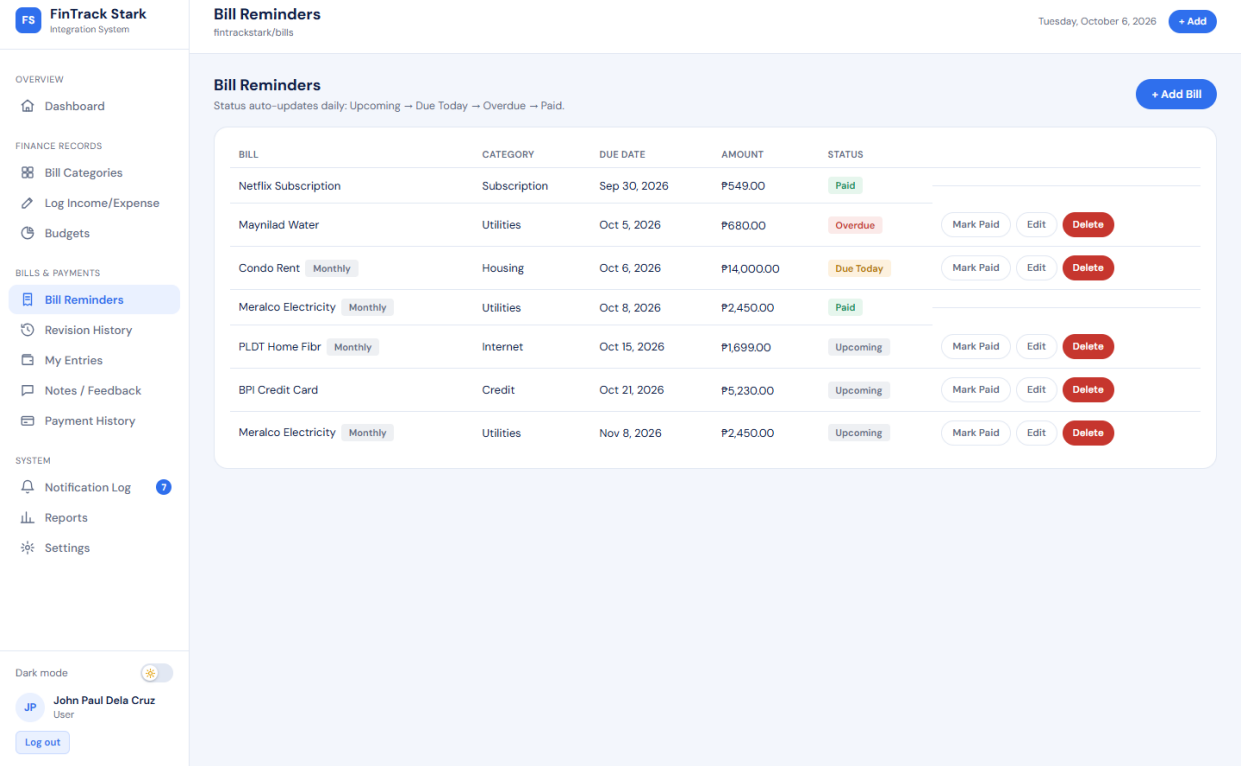
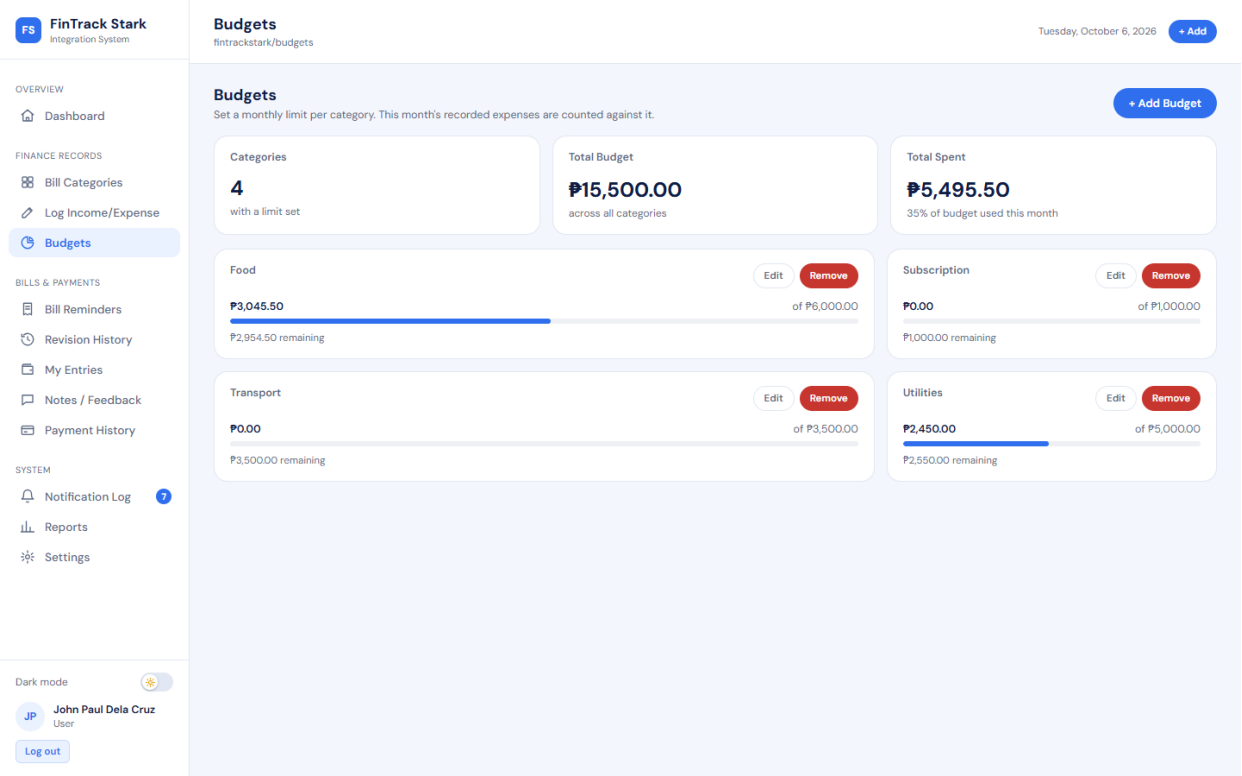


Figure 34. Bill Reminders



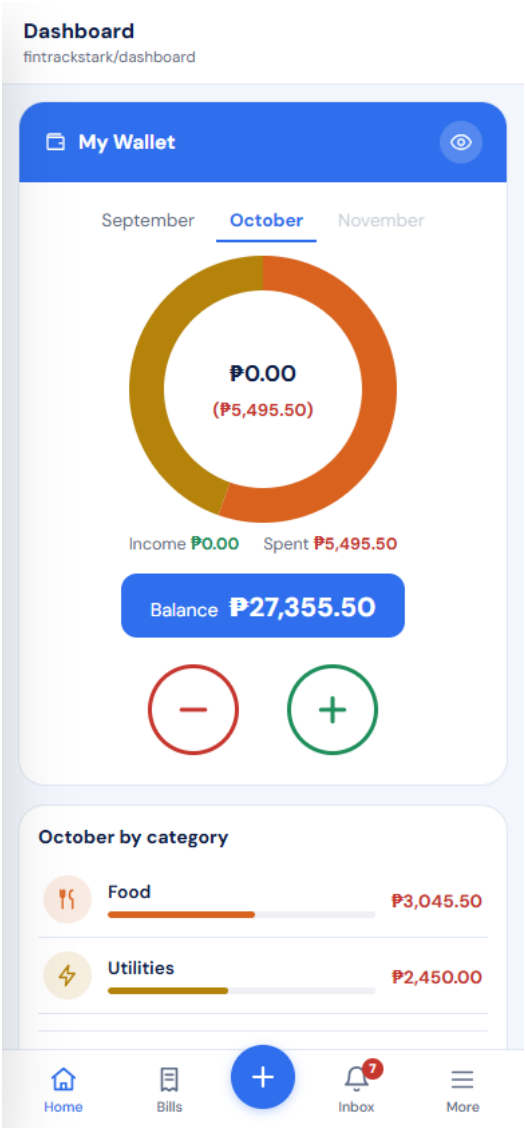


Figure 37. Phone (390 px): Home dashboard with bottom tab bar

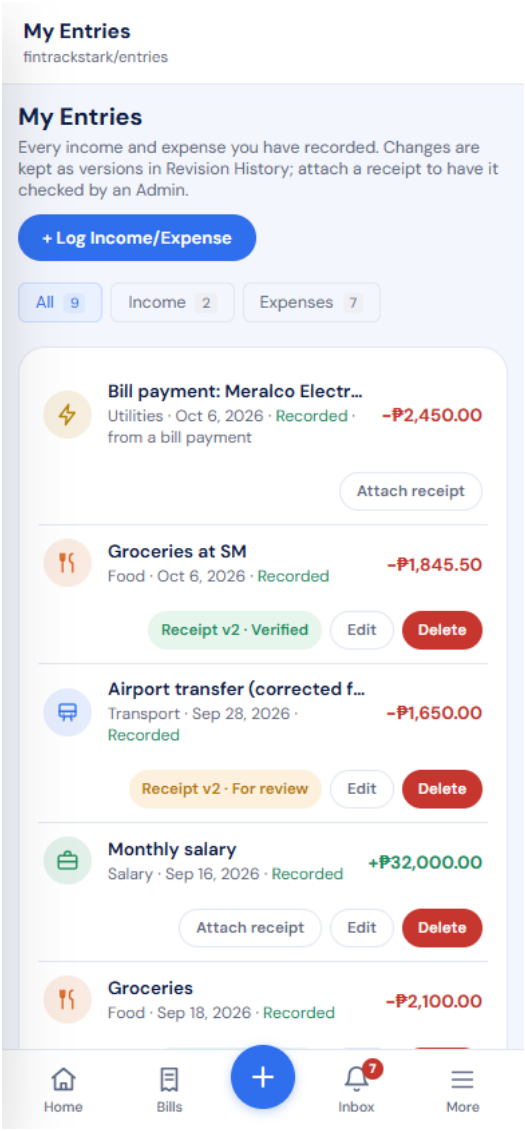


Figure 38. Phone (390 px): My Entries

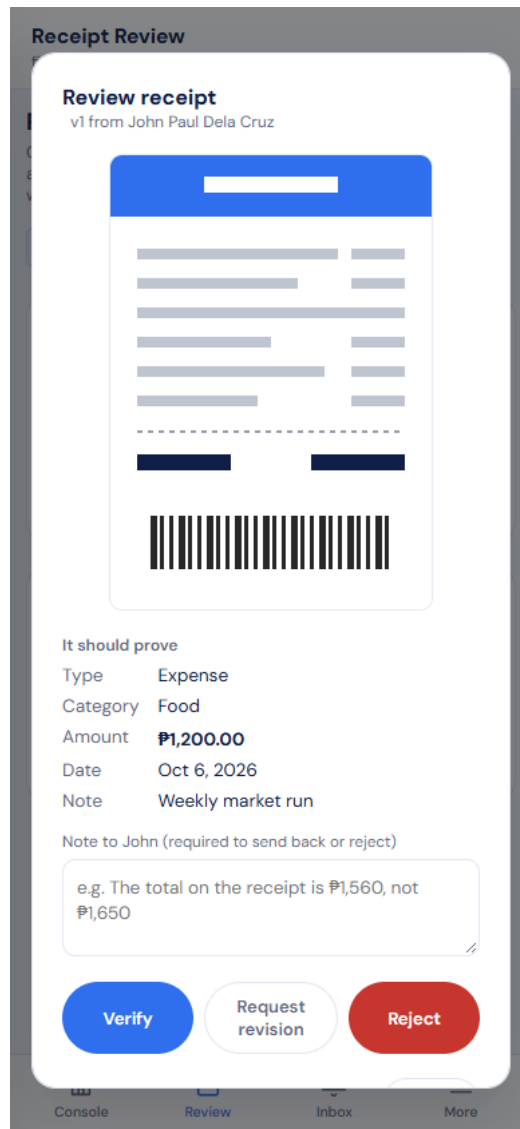


Figure 39. Phone (390 px): Admin checking a receipt

Appendix C — Complete Test Results

Table 56. All 70 automated test cases (run 10/6/2026, 20:30:32)

ID	Test case	Result	Time (s)
FT-01	A registered user can sign in through the form (FR-001)	Passed	1.7
FT-02	A user can create a bill record (FR-002)	Passed	2.4
FT-03	A user can record an income/expense entry (FR-003)	Passed	1.8
FT-04	The system tracks entry versions (FR-004)	Passed	1.7
FT-05	A user can correct and delete their own entries (FR-015)	Passed	2.3
FT-06	Notes are stored per entry (FR-006)	Passed	1.2
FT-07	The dashboard summarises balance, bills and budgets (FR-009)	Passed	1.8
FT-08	Important actions are written to the audit log (FR-010)	Passed	2.2
FT-09	Every screen has its own address	Passed	2.7
FT-10	Addresses survive a reload, and the browser's own buttons work	Passed	3.3
FT-11	A receipt photo can be attached while recording an entry (FR-003)	Passed	2.0
FT-12	The Admin can verify, reject or send back a receipt from the review page (FR-005)	Passed	1.9

ID	Test case	Result	Time (s)
IT-01	Paying a bill cascades into an expense, a notification, and an audit entry	Passed	1.7
IT-02	A notification reaches an open screen without a refresh	Passed	1.4
IT-03	Notifications reach only the person they are about	Passed	0.0
IT-04	A recorded entry reaches the dashboard at once, without a refresh	Passed	2.3
IT-05	Recorded expenses are counted against the matching budget	Passed	1.9
IT-06	The reminder service raises overdue alerts on its own schedule	Passed	3.5
IT-07	A receipt moves through review: submitted, sent back, resubmitted as v2, verified	Passed	0.2
IT-08	Changing the figures of an entry sends its verified receipt back for review	Passed	0.1
ET-01	Missing required fields are rejected with a usable message	Passed	0.0
ET-02	Invalid amounts are rejected at the boundary	Passed	0.0
ET-03	Duplicate data is refused and the reason is explained	Passed	1.9
ET-04	A malformed record id returns 400, not a crash or a hang	Passed	0.0
ET-05	An invalid workflow transition is refused	Passed	0.2
ET-06	Invalid uploads are refused with a reason, and nothing is saved	Passed	0.2
ET-07	A failed review is refused cleanly and written to the log as Failed	Passed	0.1
ST-01	Protected endpoints refuse unauthenticated callers (NFR-001)	Passed	0.0
ST-02	A user cannot reach another user's records (NFR-002)	Passed	0.2
ST-03	Privileges cannot be escalated from a lower role (section 8.2)	Passed	1.6
ST-04	Receipts are seen only by their owner and the Admin, and only the Admin decides (NFR-002, section 8.2)	Passed	0.1
E2E-01	A new account records, corrects and reports an expense	Passed	4.0
E2E-02	A receipt goes from submission to final approval	Passed	3.5
RT-01	A refused edit leaves the entry exactly as it was	Passed	0.1
RT-02	A refused payment leaves the bill unpaid and writes nothing	Passed	0.1
RT-03	Entries count at once; only receipts go through review	Passed	0.0
RT-04	Dates and field types are validated, never a 500	Passed	0.1
RT-05	An Admin keeps no wallet: finance records are for Users	Passed	0.1
RT-06	An Admin password reset forces a new password	Passed	0.6
RT-07	Security headers and JSON 404s	Passed	0.1
RT-08	The login form is not prefilled with an account	Passed	1.4
RT-09	A session that ends mid-use asks for the password and keeps the form	Passed	2.4
RT-10	A malformed address does not blank the app	Passed	1.2
RT-11	A failed save keeps the dialog open with what was typed	Passed	2.1
RT-12	The review page is the Admin's alone	Passed	1.8
RT-13	Each role's dashboard leads with its own work	Passed	1.9
RT-14	Phones get a bottom tab bar with quick add	Passed	2.1
RT-15	Categories match the entry type everywhere	Passed	0.1
RT-16	Income and expenses cannot be dated in the future	Passed	0.0
RT-17	Paying a monthly bill schedules next month's	Passed	0.2
RT-18	Crossing 80% and 100% of a budget sends a warning	Passed	0.2
RT-19	An edit can fix the type, category and date of an entry	Passed	0.1
RT-20	One request brings everything a screen needs, and only your own	Passed	0.1
RT-21	The app is installable and opens offline	Passed	1.8

ID	Test case	Result	Time (s)
RT-22	The landing page explains the system and its links work	Passed	1.5
RT-23	Bill reminders run inside the API, and never twice	Passed	0.7
RT-24	Data from the old review workflow is converted on start-up	Passed	1.4
RT-25	Password fields can be shown and hidden	Passed	2.4
RT-26	Your own action is confirmed once, not announced back as news	Passed	3.7
RT-27	Notes have a title and open in full, like notes on a phone	Passed	3.4
RT-28	A note can only be changed by the person who wrote it	Passed	0.1
RT-29	The Notification Log reads like a message list	Passed	2.1
RT-30	Amounts must be real numbers, and fields of the wrong type are refused in plain words	Passed	0.1
RT-31	An Admin cannot read another person's entries through their history	Passed	0.0
RT-32	A slow first load shows a loading message, not an empty account	Passed	4.8
RT-33	The public pages fit small phones without sideways scrolling	Passed	4.2
RT-34	A restore changes nothing unless it is confirmed	Passed	0.4
RT-35	One account cannot hold unlimited live connections open	Passed	0.1
RT-36	The Admin manages system settings; changes are validated and audited	Passed	2.2
RT-37	The settings take effect: reminder lead time, budget warning level, session length	Passed	0.7

Appendix D — Source Code Excerpts

D.1 A receipt is decided once — atomic conditional update (apps/server/src/controllers/receiptController.js)

```
// Decided once: the update only matches a receipt still waiting, so two
// Admins acting at the same moment cannot both decide it.
const decided = await Receipt.findOneAndUpdate(
  { _id: receipt._id, latest: true, status: "For Review" },
  { $set: { status: decision.status, reviewedBy: req.user.id, reviewerName: req.user.name,
    reviewedAt: new Date(), reviewNote: text } },
  { new: true }
);
if (!decided) {
  return res.status(409).json({ error: "This receipt was already decided, replaced or withdrawn." });
}
```

D.2 A file is checked by its first bytes, not its name (receiptController.js)

```
const SIGNATURES = {
  "image/jpeg": (b) => b[0] === 0xff && b[1] === 0xd8 && b[2] === 0xff,
  "image/png": (b) => b.slice(0, 8).equals(Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d, 0x0a, 0x1a,
    0x0a])),
  "image/webp": (b) => b.slice(0, 4).toString("latin1") === "RIFF" && b.slice(8, 12).toString("latin1")
    === "WEBP",
  "application/pdf": (b) => b.slice(0, 5).toString("latin1") === "%PDF-",
};
```

D.3 Refused changes become Failed lines in the integration log (apps/server/src/middleware/failureLog.js)

```
function failureLog(req, res, next) {
  if (req.method === "GET") return next();
  const json = res.json.bind(res);
  res.json = (body) => {
    if (res.statusCode >= 400 && res.statusCode !== 401 && req.user) {
      // Built from the address itself: by the time a database validation
      // error reaches the central error handler, Express has already
      // forgotten which router matched. Record ids become ":id".
    }
  };
}
```

```

    const route = req.originalUrl.split("?")[0].replace(/^\/api\/\//, "").replace(/[0-9a-f]{24}/gi,
    ":id");
    const reason = (body && body.error) || `HTTP ${res.statusCode}`;
    logFailure(req.user.name, `Failed: ${req.method} ${route}`, `${res.statusCode} - ${reason}`,
    req.params && req.params.id);
  }
  return json(body);
};
next();
}

```

D.4 The reminder service claims each bill once per day (apps/server/src/services/reminderSweep.js)

```

const toRemind = [];
for (const bill of due) {
  if (bill.lastRemindedOn === today) continue;
  const claim = await Bill.updateOne(
    { _id: bill._id, paid: false, lastRemindedOn: { $ne: today } },
    { $set: { lastRemindedOn: today } }
  );
  if (claim.modifiedCount === 1) toRemind.push(bill);
}

```

D.5 A bill is paid once — atomic claim (apps/server/src/controllers/billController.js)

```

const claimed = await Bill.findOneAndUpdate(
  { _id: bill._id, paid: false },
  { $set: { paid: true, paidOn, paidAmount: amount } },
  { new: true }
);
if (!claimed) {
  return res.status(400).json({ error: "This bill is already marked as paid." });
}

```

Appendix E — Project Folder Structure

Repository layout (main folders)

```

SIA-Final-Project/
  apps/
    client/
      public/
      src/
      index.html
      package.json
      vite.config.js
    server/
      src/
      package.json
      server.js
  docs/
    appendices/
      sample-database-backup.json
    diagrams/
    screenshots/
    ~$nTrack_Stark_Final_Project.docx
    DEPLOYMENT.md
    FinTrack_Stark_Final_Project_v2.docx
    FinTrack_Stark_Final_Project_v2.pdf
    FinTrack_Stark_Final_Project.docx
    FinTrack_Stark_Final_Project.pdf
  scripts/
    backup.js
    create-admin.js
    restore.js

```

```
sample-receipt.js
seed.js
services/
  reminder/
    index.js
    package.json
tests/
  end-to-end.spec.js
  error-handling.spec.js
  functional.spec.js
  global-setup.js
  helpers.js
  integration.spec.js
  receipt-helpers.js
  regression.spec.js
  security.spec.js
package-lock.json
package.json
playwright.config.js
README.md
```